Revisiting the Maximum Defective Clique Problem: Faster Branching and a Tighter Upper Bound

Kewu Yang
Harbin Institute of Technology, Shenzhen
China
220110422@stu.hit.edu.cn

Kaiqiang Yu
State Key Laboratory of Novel Software Technology,
Nanjing University
kaiqiang.yu@nju.edu.cn

Shengxin Liu
Harbin Institute of Technology, Shenzhen
China
sxliu@hit.edu.cn

Zhaoqun Gu
Harbin Institute of Technology, Shenzhen
China
guzhaoquan@hit.edu.cn

ABSTRACT

The k-defective clique model relaxes the strict completeness constraint of the traditional clique by allowing up to k missing edges, providing a robust formulation for detecting cohesive structures in noisy graphs. Consequently, the maximum k-defective clique problem has attracted significant attention. State-of-the-art exact algorithms predominantly adopt the branch-and-bound framework, which recursively partitions the current problem instance (or branch) into two sub-problems via a branching procedure, until each sub-problem becomes trivially solvable. However, this strategy often leads to excessive branching by overlooking intermediate sub-problems that are non-trivial yet efficiently solvable. While recent studies have attempted to refine branching procedures, they fail to address this structural redundancy. To address this, we propose `BBRes`, a framework that incorporates a novel early termination strategy into the recursive branching process. By employing a specialized polynomial-time solver to identify and resolve tractable sub-instances, `BBRes` effectively avoids redundant branching steps. Additionally, we design a tailored branching strategy that synergizes with this termination mechanism. As a result, `BBRes` achieves an improved theoretical worst-case time complexity. Furthermore, to enhance practical performance, we propose a tighter upper bound based on a novel double graph coloring method integrated with max-flow techniques, which is orthogonal to the branching framework. Extensive experiments demonstrate that `BBRes` achieves a speedup of up to 53.1× over state-of-the-art algorithms.

PVLDB Reference Format:
Kewu Yang, Kaiqiang Yu, Shengxin Liu, and Zhaoqun Gu. Revisiting the Maximum Defective Clique Problem: Faster Branching and a Tighter Upper Bound. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Thaumaturge2020/BBRes/.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Graphs are widely used to model relationships between entities in various applications, such as social media, biological science, and e-commerce. Extracting cohesive/dense subgraphs from large real-world graphs is a fundamental problem in graph analytics. Such cohesive subgraphs often carry interesting information, which can facilitate various tasks across different domains. Examples include finding communities in social networks [5, 21], detecting anomalies in financial networks or social media [2, 6, 32, 51], and discovering biologically relevant functional groups in biological networks [44].

One notable cohesive subgraph model is the clique, which requires that every pair of vertices is connected by an edge [9, 15, 20, 35, 36, 39, 41, 42]. However, the strict completeness requirement of cliques is often too restrictive for real-world applications due to data quality issues caused by noise and missing values. To solve this issue, recent studies have relaxed the clique model and proposed various clique relaxation models, including quasi-clique [30, 38, 47, 52, 53], k-plex [13, 24, 40, 46, 48, 54], k-club [7], and k-defective clique [11, 12, 14, 17, 23, 29, 34, 50]. In this paper, we focus on the k-defective clique model, which relaxes the clique model by allowing up to k missing edges, where k is a positive integer. Conceptually, a k-defective clique is a subgraph induced by a set of vertices S that contains at least $\binom{|S|}{2} - k$ edges. We study the problem of finding the maximum k-defective clique, i.e., the one with the largest number of vertices, which has been used for various applications such as interaction prediction in biological networks [50], cluster detection [18], and social network analysis [26].

State-of-the-art methods. Recently, a number of exact algorithms have been proposed in the literature for finding the maximum k-defective clique [11, 12, 14, 17, 23, 28, 29, 34]. The majority of them follow a similar *branch-and-bound (BB)* framework [11, 12, 17]. Specifically, this framework recursively partitions the problem into two sub-problems on *smaller* subgraphs via a *branching* method, continuing until each sub-problem can be solved trivially (i.e., the corresponding subgraph is already a k-defective clique). Each sub-problem corresponds to a branch. To improve efficiency, reduction rules are developed to prune unpromising branches that cannot contain the largest k-defective clique. Among these reductions, upper-bound-based pruning is often the most powerful in practice. The rationale is that the algorithm computes an upper bound on the size of any k-defective clique derivable from the current branch, and

it safely prunes the branch if this upper bound is not larger than the size of the largest k-defective clique found so far. Therefore, tight upper bounds are preferred to enhance practical performance. In terms of theoretical complexity, we note that recent works focus on sharpening the branching method to improve the worst-case time complexity. The best known complexity is $O^*(\gamma_k^n)$ [17], where O^* suppresses polynomial factors, n is the number of vertices, and γ_k is the largest real root of the equation $x^{k+3} - 2x^{k+2} + x^2 - x + 1 = 0$.

While some recent approaches, such as DnBK [34] and WODC [28], attempt to depart from the standard BB framework to reduce the exponential base of the time complexity, these theoretical improvements come at a prohibitive cost. Specifically, they introduce a substantial overhead factor scaled by $(\delta\Delta)^{\Theta(k)}$, where δ and Δ denote the degeneracy and maximum degree of the graph, respectively. In real-world graphs where Δ is typically large, this overhead explodes as k increases, rendering such methods computationally impractical for difficult instances. Consequently, existing state-of-the-art algorithms, whether adhering to the standard BB framework or alternative paradigms, still suffer from significant efficiency bottlenecks in both theory and practice.

Our method. In this paper, instead of solely refining the branching strategies as existing methods do, we focus on *both the termination strategy and the branching strategy*. Specifically, we observe that existing methods terminate the recursive branching procedure of generating new branches only when the current sub-problem can be solved *trivially*, which incurs an excessive number of recursive calls to reach such a trivial branch. As a result, they generate an excessive number of branches. Motivated by this, we propose to terminate the recursive branching procedure once the current sub-problem can be solved efficiently (e.g., in polynomial time) via a non-recursive solver, thereby generating fewer branches. We call this strategy the *early termination strategy*. To implement this strategy, we first formulate *non-trivial* input-restricted problems (of finding the largest k-defective clique) called MissingTwoDeg, which corresponds to a collection of non-trivial branches. Then, we develop a greedy method called IRSolver to solve MissingTwoDeg instances in polynomial time. Based on IRSolver, our early termination strategy halts the recursive branching and invokes IRSolver when the current branch corresponds to an input-restricted problem. Furthermore, to fully exploit the potential of our early termination strategy, we design a new branching strategy called **BS-three**. Compared to existing strategies, **BS-three** better complements the early termination strategy as it tends to generate fewer branches. With the early termination strategy and new branching strategy, our branch-and-bound algorithm called BBRes achieves the worst-case time complexity of $O^*(\lambda_k^n)$, where λ_k is the largest real root of the equation $x^{k+4} - 2x^{k+3} + x^3 - x + 1 = 0$. For example, $\lambda_k = 1.381$, 1.705, 1.867 when $k = 1, 2, 3$, respectively. We remark that, compared to the state-of-the-art method MDC [17], our BBRes further improves the worst-case time complexity from $O^*(\gamma_k^n)$ to $O^*(\lambda_k^n)$, where λ_k is *strictly smaller than* γ_k for $k \geq 1$.

In addition, to further boost practical performance, we propose a new upper bound on the size of the k-defective clique to be found in a branch, by leveraging double graph coloring and max-flow techniques. Specifically, we perform graph coloring *twice*, assigning each vertex a pair of colors. Then, based on the double-coloring information, we construct a constrained max-flow problem to compute the upper bound. Compared to existing color-based upper bounds [12, 14, 17] that also utilize graph coloring, our upper bound **UB-Double** is *practically tighter*, as it leverages more topological information by applying graph coloring twice and incorporating the max-flow technique. We remark that this upper bound is orthogonal to the branch-and-bound framework and can also be applied to existing methods [11, 12, 14, 17, 23, 29].

Contributions. We summarize our main contributions as follows.

- We propose a new branch-and-bound algorithm called BBRes which is based on our newly developed early termination strategy and branching strategy. BBRes has the worst-case time complexity of $O^*(\lambda_k^n)$, improving upon the state-of-the-art method [17] since λ_k is strictly smaller than γ_k for $k \geq 1$. (Section 4)
- We further propose a new upper bound of the size of k-defective cliques to be found in a branch based on the double graph coloring technique and the max-flow technique. Our upper bound is tighter than the existing ones in practice. (Section 5)
- Finally, we conduct extensive experiments on a collection of 139 real-world graphs. The results show that our BBRes outperforms other baselines, including the state-of-the-art methods kDC2 [12], MDC [17], DnBK [34], and WODC [28], by solving more problem instances and achieving speedups of up to 53.1×. (Section 6)

For the rest of the paper, we define the problem in Section 2, present state-of-the-art methods in Section 3, review related work in Section 7, and conclude the paper in Section 8.

2 PRELIMINARIES

We consider an unweighted and undirected graph $G = (V, E)$, where V is the vertex set and E is the edge set. Let $n = |V|$ and $m = |E|$ denote the number of vertices and edges, respectively. We define $\overline{E}$ as the set of edges that are missing in G (referred to as **non-edges**), i.e., $\overline{E} = \{(u, v) \in V \times V \mid u \neq v \text{ and } (u, v) \notin E\}$.

Given a vertex v in V, let $N_G(v)$ (resp. $\overline{N}_G(v)$) be the set of neighbors (resp. non-neighbors) of v in G; formally, $N_G(v) = \{u \in V \mid (u, v) \in E\}$ (resp. $\overline{N}_G(v) = \{u \in V \mid (u, v) \in \overline{E}\}$). Accordingly, we define the degree of v in G as $d_G(v) = |N_G(v)|$, and its non-degree as $\overline{d}_G(v) = |\overline{N}_G(v)|$. We note that a vertex is neither a neighbor nor a non-neighbor of itself. Given a vertex subset $S \subseteq V$, we use $G[S]$ to denote the subgraph of G induced by S, i.e., $G[S]$ consists of the set of vertices S and the set of edges $\{(u, v) \in E \mid u, v \in S\}$. All subgraphs considered in this paper are induced subgraphs. Given a subgraph g of G, we use $V(g)$, $E(g)$, and $\overline{E}(g)$ to denote its sets of vertices, edges, and non-edges, respectively.

Abbreviations. For simplicity, we omit the subscript G from the notation when the context is clear. Furthermore, for $S \subseteq V$, we abbreviate $N_{G[S]}(\cdot)$ as $N_S(\cdot)$, $d_{G[S]}(\cdot)$ as $d_S(\cdot)$, and $E(G[S])$ as $E(S)$. Similarly, we abbreviate $\overline{N}_{G[S]}(\cdot)$, $\overline{d}_{G[S]}(\cdot)$, and $\overline{E}(G[S])$ as $\overline{N}_S(\cdot)$, $\overline{d}_S(\cdot)$, and $\overline{E}(S)$, respectively.

In this paper, we focus on the concept of k-defective clique.

Definition 2.1 (k-Defective Clique [50]). A graph g is said to be a *k-defective clique* if it contains at most k non-edges, i.e., $|\overline{E}(g)| \leq k$ or equivalently, $|E(g)| \geq \frac{|V(g)|(|V(g)|-1)}{2} - k$.

Clearly, a 0-defective clique is simply a clique, in which every pair of vertices is adjacent. In addition, we note that the k-defective clique satisfies the *hereditary* property, i.e., any subgraph of a k-defective clique is also a k-defective clique [37]. We now formally state the problem studied in this paper.

Problem Definition (Maximum k-Defective Clique [11, 12, 14, 17, 23, 34]). *Given a graph $G = (V, E)$ and a positive integer k, the* maximum k-defective clique problem *aims to find the maximum k-defective clique in G, i.e., the k-defective clique with the largest number of vertices.*

We note that the maximum k-defective clique in G may not be unique, and the maximum k-defective clique problem is known to be NP-hard [33, 43].

3 STATE-OF-THE-ART ALGORITHMS

The state-of-the-art algorithms for the maximum k-defective clique predominantly adopt the *branch-and-bound* (BB) framework [11, 12, 17]. The core idea is to recursively partition the current problem instance (i.e., search space), which aims to find the largest k-defective clique, into two smaller sub-problems via *branching* until each of them can be solved trivially. Specifically, a problem instance (or branch) is represented as a triple (g, S, C), where:

- Graph g is a subgraph of the input graph G induced by the vertex set $S \cup C$, i.e., $g = G[S \cup C]$;
- Partial set S is a set of vertices that induces a k-defective clique and must be contained in the largest k-defective clique found within this branch;
- Candidate set C is a set of vertices that will be considered to be included in S.

Solving a branch (g, S, C) means finding in this branch the largest k-defective clique g^* that (1) includes all vertices in S and (2) is a subgraph of g, i.e., $S \subseteq V(g^*)$ and $V(g^*) \subseteq C \cup S$. Clearly, solving the branch $(G, \emptyset, V)$ finds the maximum k-defective clique in G.

To solve a branch (g, S, C), the framework recursively partitions the branch into two sub-branches via branching. In particular, it selects a vertex v^* called *pivot* from the candidate set C, and generates two sub-branches. The first one $(g_1, S \cup \{v_p\}, C \setminus \{v_p\})$ removes the pivot v_p from C to S (which aims to explore the largest k-defective clique that includes v_p); the second one $(g_2, S, C \setminus \{v_p\})$ discards the pivot v_p from C (which aims to explore the largest k-defective clique that excludes v_p). Here, g_1 and g_2 are the subgraphs induced by the updated partial and candidate sets in each sub-branch. By recursively solving both sub-branches, the framework ensures that all possibilities are explored and thus solves the original branch.

Summary. We summarize the BB framework in Algorithm 1. Specifically, the algorithm maintains the largest k-defective clique g^* seen so far during the recursion (Line 1). To boost the practical performance, it utilizes ***the diameter-two property*** ([14]) of large k-defective cliques (i.e., any k-defective clique with at least $k + 2$ vertices has diameter at most 2) and involves the following stages.

- **Stage-I** adopts the diameter-two property for pruning by assuming that the largest k-defective clique is of size at least $k + 2$ (Lines 2-5). Specifically, it first computes a degeneracy ordering of the vertices in G (Line 2), which can be done efficiently in $O(m)$ by the peeling algorithm [4]. Then, it divides the problem of finding the maximum k-defective clique into n sub-problems (Lines 3-5). The i-th one aims to find the maximum k-defective clique that (1) includes v_i and (2) is a subgraph of $G[\{v_i, v_{i+1}, ..., v_n\}]$ by invoking the BB procedure with the branch $(G_{v_i}, \{v_i\}, V(G_{v_i}) \setminus \{v_i\})$ (Lines 3-5). In particular, G_{v_i} is the subgraph of G induced by v_i's two-hop neighbors in $\{v_i, v_{i+1}, ..., v_n\}$, i.e., $N^{\leq 2}(v_i) \cap \{v_i, v_{i+1}, \ldots, v_n\}$ (note that a k-defective clique with at least $k + 2$ vertices has diameter of at most 2 and thus the largest one containing v_i is a subset of $N^{\leq 2}(v_i)$). Clearly, if the found largest k-defective clique g^* is of at least $k + 1$ vertices, g^* is guaranteed to be a maximum k-defective clique of G.
- **Stage-II** continues the search for the maximum k-defective clique when Stage-I does not succeed, i.e., when g^* obtained in Stage-I has fewer than $k + 1$ vertices (Line 6). To this end, it invokes the BB procedure with the branch $(G, \emptyset, V)$.

Algorithm 1: The state-of-the-art branch-and-bound (BB) framework [11, 12, 17]

Input: A graph $G = (V, E)$ and a positive integer k
Output: The maximum k-defective clique g^*

```
/* Stage-I: With the diameter-two property                    */
1  Let g* ← ∅ be the largest k-defective clique seen so far;
2  Let V = {v_1, v_2, ..., v_n} be a degeneracy ordering of vertices in G;
3  foreach v_i ∈ {v_1, v_2, ···, v_n} do
4      G_{v_i} ← the subgraph of G induced by
         N^{≤2}(v_i) ∩ {v_i, v_{i+1}, ..., v_n} ;
5      BB_Rec(G_{v_i}, {v_i}, V(G_{v_i}) \ {v_i});
/* Stage-II: Without the diameter-two property                 */
6  if |V(g*)| < k + 1 then BB_Rec(G, ∅, V);
7  return g*;
8  Procedure BB_Rec(g, S, C)
       /* Reduction                                             */
9      UB ← an upper bound of the branch;
10     if UB ≤ |V(g*)| then return;
11     Refine C (and g) by applying reduction rules;
       /* Termination                                           */
12     if g is a k-defective clique then
13         if |V(g)| > |V(g*)| then g* ← g;
14         return;
       /* Branching                                             */
15     v_p ← a pivot selected from C;
16     Construct g_1 and g_2 based on v_p, S, and C;
17     BB_Rec(g_1, S ∪ {v_p}, C \ {v_p});
18     BB_Rec(g_2, S, C \ {v_p});
```

In addition, the BB procedure called BB_Rec is summarized in Lines 8-18. We note that the recursive procedure terminates when g becomes a k-defective clique since g is the largest k-defective clique in the branch (Lines 12-14). To reduce the search space, the framework employs two techniques as follows.

- **Upper-bound-based reductions** (Lines 9-10). It first computes an upper bound of the size of the largest k-defective clique in the branch. Then, the branch can be terminated if the upper bound is no larger than the largest k-defective clique seen so far.

- **Pivot-based branching** (Lines 15-18). The number of branches generated by the pivot-based branching depends on the pivot selection strategy (Line 15). We note that recent studies have explored various strategies for pivot selection to reduce the number of branches [11, 12, 17].

For brevity, we do not review the detailed implementation of these techniques in state-of-the-art algorithms.

Time complexity. Following Algorithm 1, the state-of-the-art studies have focused on improving the worst-case time complexity by sharpening the pivot selection strategy [11, 12, 17]. As a result, the latest algorithm has the time complexity of $O^*(\gamma_k^n)$ [17], where O^* ignores the polynomial factors and γ_i is the largest real root of the equation $x^{i+3} - 2x^{i+2} + x^2 - x + 1 = 0$.

Remark. It is worth noting that two recent algorithms, DnBK [34] and WODC [28], employ frameworks distinct from the above BB framework to successfully reduce the exponential base in time complexity. However, this improvement comes at the cost of introducing a substantial overhead factor scaled by $(\delta\Delta)^{\Theta(k)}$, where δ is the degeneracy of the input graph G and Δ is the maximum degree of G. This factor becomes prohibitively large when k is not a constant, severely limiting scalability. As demonstrated in our experiments (Section 6), both methods are less competitive compared to our proposed algorithm. We defer the detailed complexity analysis of both algorithms to the related work section (Section 7).

4 A NEW BRANCH-AND-BOUND FRAMEWORK: BBRES

4.1 BBRes: Motivation and Overview

Motivation. The existing BB framework recursively partitions the current problem instance (g, S, C) into two sub-problems via branching until each of them can be solved *trivially*, i.e., g becomes a k-defective clique. As a result, the branching procedure, which is biased towards generating trivial instances, produces an exponentially large number of branches in the worst case (e.g., $O(\gamma_k^n)$ [17]), thereby dominating the time complexity. To reduce the number of branches, we propose to *guide the branching procedure towards non-trivial yet solvable problem instances*. The rationale is to terminate the recursive branching procedure early whenever the current problem instance can be solved efficiently (e.g., in polynomial time) by a dedicated solver, thus generating fewer branches.

Overview. Motivated by the above, we develop a new BB framework called BBRes, as summarized in Algorithm 2. BBRes differs from Algorithm 1 mainly in the *early termination strategy* (Lines 12-15) and the *branching strategy* (Lines 16-18). Specifically, we first formulate an input-restricted problem instance (i.e., a branch (g, S, C) satisfying certain constraints), which is *non-trivial* (i.e., g can be a non-k-defective clique). Note that such input-restricted instances will be solved recursively via branching in Algorithm 1. However, we observe that these instances can be solved efficiently in polynomial time by a non-recursive method called IRSolver in Section 4.2. Based on this observation, our BBRes can terminate the recursive branching procedure once it reaches an input-restricted instance, i.e., satisfying the termination conditions, and uses IRSolver to solve the branch. Furthermore, we observe that the existing branching strategies are not suitable for our BBRes since

Algorithm 2: Our framework BBRes

Input: A graph $G = (V, E)$ and a positive integer k
Output: The maximum k-defective clique g^*

```
   /* Stage-I: With the diameter-two property                    */
1  Let g* ← ∅ be the largest k-defective clique seen so far;
2  Let V = {v_1, v_2, ..., v_n} be a degeneracy ordering of vertices in G;
3  foreach v_i ∈ {v_1, v_2, ···, v_n} do
4      G_{v_i} ← the subgraph of G induced by
           N^{≤2}(v_i) ∩ {v_i, v_{i+1}, ..., v_n} ;
5      BBRes_Rec(G_{v_i}, {v_i}, V(G_{v_i}) \ {v_i});
   /* Stage-II: Without the diameter-two property                */
6  if |V(g*)| < k + 1 then BBRes_Rec(G, ∅, V);
7  return g*;
8  Procedure BBRes_Rec(g, S, C)
       /* Reductions (Section 5)                                 */
9      UB ← an upper bound of the branch;
10     if UB ≤ |V(g*)| then return;
11     Refining C (and g) by applying reduction rules;
       /* Early Termination Strategy (Section 4.2)               */
12     if g satisfies the early termination conditions, i.e., Condition 1
           or Condition 2 in Section 4.2 then
13         g_opt ←IRSolver(g, S, C);
14         if |V(g_opt)| > |V(g*)| then g* ← g_opt;
15         return;
       /* Branching (Section 4.3)                                */
16     v_p ← a pivot selected from C based on our new strategy;
17     BBRes_Rec(g_1, S ∪ {v_p}, C \ {v_p});
18     BBRes_Rec(g_2, S, C \ {v_p});
```

they are designed based on the framework of recursively branching to trivial branches. To boost the performance of BBRes, we further propose a new branching strategy in Section 4.3. With this newly-designed branching strategy, BBRes achieves better worst-case time complexity. In addition, we remark that we propose a novel upper bound (Lines 9-11) based on graph coloring and max-flow techniques (details will be discussed in Section 5), which is orthogonal to the framework and further enhances practical performance. We remark that our BBRes also employs those existing techniques, including reductions and pre-processing techniques for reducing the input graph [12, 17, 28], that are orthogonal to the framework.

4.2 BBRes: Early Termination Strategy

Consider a problem instance (g, S, C), i.e., finding the largest k-defective clique that is a subgraph of g and contains all vertices in S. Note that a problem instance (g, S, C) is said to be *trivial* if g is a k-defective clique. We then define the input-restricted problem instances, namely MissingTwoDeg, as follows.

Definition 4.1 (MissingTwoDeg problem instance). A problem instance (g, S, C) is said to be an input-restricted problem or MissingTwoDeg problem if it satisfies one of the following conditions:

- **Condition 1**. g is a k-defective clique, i.e., $|\overline{E}(g)| \leq k$;
- **Condition 2**. g is not a k-defective clique and for any vertex v in C, vertex v has at most 2 non-neighbors in C, i.e., $|\overline{E}(g)| > k$ and $\forall v \in C, \overline{d}_C(v) \leq 2$.

Algorithm 3: Our greedy method `IRSolver`

Input: A MissingTwoDeg problem instance (g, S, C) and a positive integer k

Output: The maximum k-defective clique g_{opt} that is a subgraph of g and contains S

1 **if** *g is a k-defective clique* **then return** g as g_{opt} directly;
2 $S_{opt} \leftarrow S, C_{temp} \leftarrow C$;
3 **while** *true* **do**
4 $\quad \Gamma_{min} \leftarrow \{v \in C_{temp} \mid \forall w \in C_{temp}, \overline{d}_{S_{opt}}(v) \leq \overline{d}_{S_{opt}}(w)\}$;
5 $\quad$ **if** ***Case 1***: $\min_{v \in \Gamma_{min}} \overline{d}_{C_{temp}}(v) \leq 1$ **then**
6 $\quad\quad v^* \leftarrow \arg\min_{v \in \Gamma_{min}} \overline{d}_{C_{temp}}(v)$;
7 $\quad$ **else if** ***Case 2***: $\min_{v \in \Gamma_{min}} \overline{d}_{C_{temp}}(v) = 2$ **then**
8 $\quad\quad v^* \leftarrow \arg\min_{v \in \Gamma_{min}} \overline{d}_{\Gamma_{min}}(v)$;
9 $\quad$ **if** $G[S_{opt} \cup \{v^*\}]$ *is not a k-defective clique* **then break**;
10 $\quad C_{temp} \leftarrow C_{temp} \setminus \{v^*\}; S_{opt} \leftarrow S_{opt} \cup \{v^*\}$;
11 **return** $G[S_{opt}]$ as the solution g_{opt};

Clearly, we note that MissingTwoDeg problem instances are non-trivial since g could not be a k-defective clique. Therefore, terminating the branching procedure at the MissingTwoDeg problem instances will reduce the number of generated branches. One remaining question is how to solve a MissingTwoDeg problem efficiently in polynomial time. To solve this, we then introduce a greedy method called `IRSolver` as follows.

Overview of `IRSolver`. If g is a k-defective clique, we can solve the problem by returning g directly. Otherwise, `IRSolver` solves the problem in a greedy manner, which runs in multiple rounds, as summarized in Algorithm 3. Specifically, it maintains two sets S_{opt} (initially as S) and C_{temp} (initially as C). The key idea is to iteratively select a vertex v^* from C_{temp} and move it from C_{temp} to S_{opt} while maintaining that $G[S_{opt}]$ is a k-defective clique until this is no longer possible (Lines 3-10, details will be discussed later). Finally, it returns $G[S_{opt}]$ as the solution (Line 11). We then elaborate on the details regarding the greedy strategy of selecting v^*, the correctness, and the time complexity analysis.

Greedy strategy of `IRSolver`. At each round (Lines 3-10), `IRSolver` selects a vertex v^* from C_{temp} based on the following greedy strategy. Specifically, let Γ_{min} be the set of vertices each of which has the smallest number of non-neighbors in S_{opt}, formally,

$$\Gamma_{min} = \{v \in C_{temp} \mid \forall w \in C_{temp}, \overline{d}_{S_{opt}}(v) \leq \overline{d}_{S_{opt}}(w)\}. \quad (1)$$

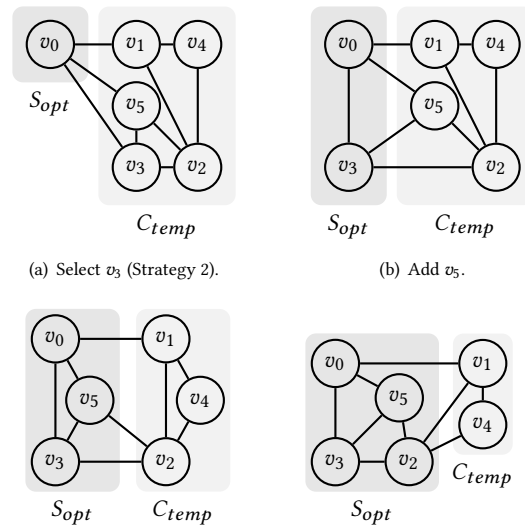
Then, we select vertex v^* from Γ_{min}. In general, there are two cases.

- **Case 1:** $\min_{v \in \Gamma_{min}} \overline{d}_{C_{temp}}(v) \leq 1$, i.e., there exists a vertex v in Γ_{min} that has the number of non-neighbors in C_{temp} smaller than 2. In this case, we select from Γ_{min} the vertex with the smallest number of non-neighbors in C_{temp} as v^*, formally,

$$\text{Greedy strategy 1: } v^* \leftarrow \arg\min_{v \in \Gamma_{min}} \overline{d}_{C_{temp}}(v). \quad (2)$$

- **Case 2:** $\min_{v \in \Gamma_{min}} \overline{d}_{C_{temp}}(v) = 2$, i.e., each vertex in Γ_{min} has two non-neighbors in C_{temp}. Then, we select from Γ_{min} the vertex with the smallest number of non-neighbors in Γ_{min}, formally,

$$\text{Greedy strategy 2: } v^* \leftarrow \arg\min_{v \in \Gamma_{min}} \overline{d}_{\Gamma_{min}}(v). \quad (3)$$



(a) Select v_3 (Strategy 2). (b) Add v_5. (c) Add v_2. (d) Termination.

Figure 1: Execution trace of `IRSolver` with $k = 1$. Dark and light gray regions denote S_{opt} and C_{temp}, respectively. The algorithm resolves a tie using Greedy Strategy 2 in (a), sequentially adds unique candidates in (b)-(c), and terminates in (d) as adding v_1 violates the k-defective constraint.

We note that every vertex in C_{temp} has at most two non-neighbors in C_{temp} based on the definition of the MissingTwoDeg problem. Thus, the above strategy covers all possible cases. In addition, we remark that if there are multiple choices when selecting v^* in Equation (2) or Equation (3), e.g., more than one vertex in Γ_{min} has the smallest number of non-neighbors in C_{temp} or Γ_{min}, v^* can be chosen arbitrarily from among them.

Example of `IRSolver`. Figure 1 demonstrates `IRSolver` on a `MissingTwoDeg` instance with $k = 1$, initialized with $S_{opt} = \{v_0\}$. In Figure 1(a), the candidate set is $\Gamma_{min} = \{v_1, v_3, v_5\}$. Since $\overline{d}_{C_{temp}}(v) = 2$ for all candidates, Greedy Strategy 2 (Equation (3)) is triggered, selecting v_3 for its minimum $\overline{d}_{\Gamma_{min}}$. Subsequently, unique candidates v_5 and v_2 are directly added to S_{opt} in Figure 1(b)-(c). The process terminates in Figure 1(d) as adding v_1 violates the k-defective constraint, yielding $G[S_{opt}]$ as the solution.

Correctness of `IRSolver`. The correctness of `IRSolver` can be guaranteed by the following lemma.

Lemma 4.2. *Given a MissingTwoDeg problem instance (g, S, C), `IRSolver` finds the largest k-defective clique g_{opt} that is a subgraph of g and contains all vertices in S.*

Proof. Note that if g is a k-defective clique, the problem can be directly solved by returning g (Line 1). In the following, we assume that g is not a k-defective clique. Clearly, `IRSolver` can terminate at Line 9 since $S \cup C$ is a superset of S_{opt} and is not a k-defective clique. Thus, assume that `IRSolver` runs in $\tau + 1$ rounds ($\tau \geq 1$). At the i-th ($1 \leq i \leq \tau$) round, it adds v_i^* to S_{opt}. Finally, it terminates at the $(\tau+1)$-st round since $S \cup \{v_1^*, ..., v_{\tau+1}^*\}$ is no longer a k-defective

clique. In addition, for $0 \le i \le \tau + 1$, we let

$$S^i_{opt} = S \cup \{v^*_1, ..., v^*_i\} \text{ and } C^i_{temp} = C \setminus \{v^*_1, ..., v^*_i\}. \quad (4)$$

$$\Gamma^i_{min} = \{v \in C^i_{temp} \mid \forall w \in C^i_{temp}, \overline{d}_{S^i_{opt}}(v) \le \overline{d}_{S^i_{opt}}(w)\} \quad (5)$$

Note that S^0_{opt} and C^0_{temp} are S and C, respectively.

We first show that, when IRSolver terminates, $g_{opt} = G[S^\tau_{opt}]$ is a *maximal k-defective clique* in g, i.e., any vertex in $C \setminus V(g_{opt})$ cannot be added to g_{opt} to form a larger k-defective clique. The reasons are as follows. Consider the last round of IRSolver where $G[S^\tau_{opt} \cup \{v^*_{\tau+1}\}]$ is no longer a k-defective clique. We note that $v^*_{\tau+1}$ has the smallest number of non-neighbors in S^τ_{opt} among other vertices in C^τ_{temp} (since it is selected from Γ_{min}). Therefore, for any vertex v in C^τ_{temp}, $G[S^\tau_{opt} \cup \{v\}]$ is not a k-defective clique since $|\overline{E}(S^\tau_{opt} \cup \{v\})| = |\overline{E}(S^\tau_{opt})| + \overline{d}_{S^\tau_{opt}}(v) \ge |\overline{E}(S^\tau_{opt})| + \overline{d}_{S^\tau_{opt}}(v^*_{\tau+1}) > k$.

We then show that g_{opt} is the largest k-defective clique in g that contains S by contradiction. Note that g_{opt} contains S clearly since S_{opt} is a superset of S. Let g_{sol} be the largest k-defective clique in g that contains S. Assume that g_{opt} is not the largest k-defective clique that contains S, i.e., $|V(g_{sol})| > |V(g_{opt})|$. Then, we will show that, by the following construction process starting with g_{sol}, there exists a largest k-defective clique g' containing S such that g_{opt} is a subgraph of g', i.e., $V(g_{opt}) \subset V(g')$, which contradicts to the fact that g_{opt} is a maximal k-defective clique in g.

Finally, to complete the proof, we introduce an iterative process to construct g' based on g_{opt} and g_{sol}. Let $\langle v^*_1, v^*_2, ..., v^*_\tau \rangle$ be the ordering of vertices in $V(g_{opt}) \setminus S$. The procedure has four steps.

- **Step 1: Initialization.** Initialize g'_0 to be g_{sol} and i to be 0;
- **Step 2: Termination checking.** If g'_i contains all vertices in g_{opt}, i.e., $V(g'_i) \supseteq V(g_{opt})$, set $g' = g'_i$ and stop the process.
- **Step 3: Construction.** Find the vertex $v^*_{o_i}$ with the smallest index o_i ($1 \le o_i \le \tau$) that is in $V(g_{opt}) \setminus S$ but not in g'_i. We can easily deduce that g'_i contains all vertices in $S^{o_i-1}_{opt}$ and $V(g'_i) \setminus S^{o_i-1}_{opt}$ is a subset of $C^{o_i-1}_{temp}$, formally,

$$S^{o_i-1}_{opt} \subseteq V(g'_i) \text{ and } V(g'_i) \setminus S^{o_i-1}_{opt} \subseteq C^{o_i-1}_{temp}. \quad (6)$$

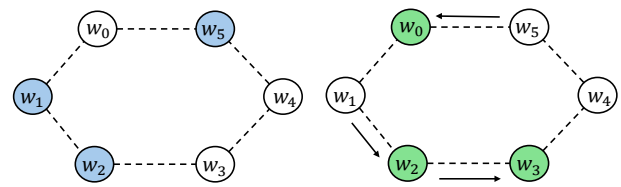
  Based on the above, we have

$$V(g'_i) = S^{o_i-1}_{opt} \cup \Psi_i \text{ where } \Psi_i = V(g'_i) \setminus S^{o_i-1}_{opt}. \quad (7)$$

  We then have three cases to construct g'_{i+1} (note that $\overline{d}_{\Psi_i}(v^*_{o_i}) \le \overline{d}_C(v^*_{o_i}) \le 2$ based on Definition 4.1).

  - **Case 1:** $\overline{d}_{\Psi_i}(v^*_{o_i}) \le 1$. Let v be an arbitrary vertex in Ψ_i if $\overline{d}_{\Psi_i}(v^*_{o_i}) = 0$; otherwise if $\overline{d}_{\Psi_i}(v^*_{o_i}) = 1$, let v be $v^*_{o_i}$'s non-neighbor in Ψ_i. We construct g'_{i+1} by swapping vertex v with $v^*_{o_i}$, formally, $g'_{i+1} = g[V(g'_i) \setminus \{v\} \cup \{v^*_{o_i}\}]$. Clearly, g'_{i+1} is a k-defective clique since $|\overline{E}(g'_{i+1})| = |\overline{E}(g'_i)| - \overline{d}_{g'_i}(v) + \overline{d}_{V(g'_i)\setminus\{v\}}(v^*_{o_i}) \le |\overline{E}(g'_i)| - \overline{d}_{S^{o_i-1}_{opt}}(v) + \overline{d}_{S^{o_i-1}_{opt}}(v^*_{o_i}) + \overline{d}_{\Psi_i\setminus\{v\}}(v^*_{o_i}) \le k$ (note that $\overline{d}_{\Psi_i\setminus\{v\}}(v^*_{o_i})$ is clearly equal to 0 and $v^*_{o_i}$ has the smallest number of non-neighbors in $S^{o_i-1}_{opt}$ among other vertices in $C^{o_i-1}_{temp}$ based on our greedy strategy).
  - **Case 2:** $\overline{d}_{\Psi_i}(v^*_{o_i}) = 2$ and $\overline{d}_{\Gamma^{o_i-1}_{min}}(v^*_{o_i}) \le 1$. In this case, all $v^*_{o_i}$'s non-neighbors in $C^{o_i-1}_{temp}$ are in Ψ_i (note that both Ψ_i and $\Gamma^{o_i-1}_{min}$ are subsets of $C^{o_i-1}_{temp}$). Let u be a $v^*_{o_i}$'s non-neighbor in Ψ_i but not in $\Gamma^{o_i-1}_{min}$ (note that such a vertex u exits based on the condition of this case). We construct g'_{i+1} by swapping vertex u with $v^*_{o_i}$, formally, $g'_{i+1} = g[V(g'_i) \setminus \{u\} \cup \{v^*_{o_i}\}]$. We can deduce that g'_{i+1} is a k-defective clique since (1) $\overline{d}_{S^{o_i-1}_{opt}}(u)$ is strictly larger than $\overline{d}_{S^{o_i-1}_{opt}}(v^*_{o_i})$ (note that vertices in $\Gamma^{o_i-1}_{min}$ has the smallest number of non-neighbors in $S^{o_i-1}_{opt}$ and $v^*_{o_i}$ is selected from $\Gamma^{o_i-1}_{min}$ based on our greedy strategy while u is not in $\Gamma^{o_i-1}_{min}$) and (2) $|\overline{E}(g'_{i+1})| = |\overline{E}(g'_i)| - \overline{d}_{g'_i}(u) + \overline{d}_{V(g'_i)\setminus\{u\}}(v^*_{o_i}) \le |\overline{E}(g'_i)| - \overline{d}_{S^{o_i-1}_{opt}}(u) + \overline{d}_{S^{o_i-1}_{opt}}(v^*_{o_i}) + \overline{d}_{\Psi_i\setminus\{u\}}(v^*_{o_i}) = |\overline{E}(g'_i)| - \overline{d}_{S^{o_i-1}_{opt}}(u) + \overline{d}_{S^{o_i-1}_{opt}}(v^*_{o_i}) + 1 \le k$.
  - **Case 3:** $\overline{d}_{\Psi_i}(v^*_{o_i}) = 2$ and $\overline{d}_{\Gamma^{o_i-1}_{min}}(v^*_{o_i}) = 2$. In this case, any vertex v in $\Gamma^{o_i-1}_{min}$ has two non-neighbors in $\Gamma^{o_i-1}_{min}$ based on our greedy strategy 2 and is adjacent to all vertices in $C \setminus \Gamma^{o_i-1}_{min}$ based on Definition 4.1. Clearly, there exists a circle in the complementary graph of $g[\Gamma^{o_i-1}_{min}]$ that contains $v^*_{o_i}$. W.l.o.g., let $\langle w_0, w_1, ..., w_c = w_0 \rangle$ ($c \ge 2$) be the circle where vertex w_j is not adjacent to w_{j+1} ($0 \le j \le c-1$) in g, and let $w_0 = v^*_{o_i}$. Let $\{w_{x_1}, w_{x_2}, ..., w_{x_t}\}$ be the vertices in $\{w_0, ..., w_c\} \cap \Psi_i$, as illustrated in Figure 2. Clearly, $w_0 = w_c$ is not in $\{w_{x_1}, w_{x_2}, ..., w_{x_t}\}$. We construct g'_{i+1} by swapping the set of vertices $\{w_{x_1}, w_{x_2}, ..., w_{x_t}\}$ with $\{w_{x_1+1}, w_{x_2+1}, ..., w_{x_t+1}\}$, i.e., $g'_{i+1} = g[V(g'_i) \setminus \{w_{x_1}, w_{x_2}, ..., w_{x_t}\} \cup \{w_{x_1+1}, w_{x_2+1}, ..., w_{x_t+1}\}]$. We can deduce that w_0 is in $\{w_{x_1+1}, w_{x_2+1}, ..., w_{x_t+1}\}]$ since w_0's non-neighbors w_1 and w_{c-1} are both in Ψ_i in this case. Therefore, g'_{i+1} contains $v^*_{o_i}$. Besides, we can easily imply that g'_{i+1} is a k-defective clique since $\{w_{x_1}, w_{x_2}, ..., w_{x_t}\}$ and $\{w_{x_1+1}, w_{x_2+1}, ..., w_{x_t+1}\}$ share the same topological structure in g, and we omit the details for the simplicity.
- **Step 4: Repetition.** Increase i by one and go to **Step 2**.

Clearly, the invariants $o_i < o_{i+1}$ and $|V(g'_i)| = |V(g_{i+1})'| = |V(g_{sol})|$ are kept during the above construction. Therefore, it can terminate and find the largest k-defective clique g' in g that contains g_{opt} and has more vertices than g_{opt}, i.e., $V(g_{opt}) \subset V(g')$. □



(a) Illustrating $\{w_{x_1}, ..., w_{x_t}\}$ (b) Illustrating $\{w_{x_1+1}, ..., w_{x_t+1}\}$

Figure 2: An example to illustrate Case 3 in the proof of Lemma 4.2 with $c = 6$ and $\{w_{x_1}, ..., w_{x_t}\} = \{w_1, w_2, w_5\}$ (indicated in blue); we have $\{w_{x_1+1}, ..., w_{x_t+1}\} = \{w_2, w_3, w_6 = w_0\}$ (indicated in green).

Time complexity of IRSolver. The time complexity of IRSolver is bounded by $O(|C|^2)$. To be specific, IRSolver maintains the

degree information $\overline{d}_{S_{opt}}(\cdot)$ and $\overline{d}_{C_{temp}}(\cdot)$ of all vertices in C during the iteration, and each round needs $O(1)$ for the update due to the removal of v^*. `IRSolver` runs in $O(|C|)$ rounds. At each round, our greedy strategy 1 can be done in $O(1)$ by using the linear heap to maintain the degree information. However, counting the non-neighbors of all vertices runs in $O(|C|^2)$, which dominates the time complexity. Therefore,the total time complexity is $O(|C|^2)$.

4.3 BBRes: New Branching Strategy

Motivation. Consider the branching procedure at a branch (g, S, C). Recent studies have shown that the performance of the proposed method depends on the pivot selection strategy used during branching. We first briefly revisit the existing pivot selection strategy [12]. Specifically, it selects from C the vertex that has at least one non-neighbor in S as the pivot v_p; if no such vertex exists, it selects from C an arbitrary vertex as the pivot v_p. We observe that the existing branching strategy [12] could select those vertices that have at most two non-neighbors in C as the pivot. We note that those vertices can be skipped when selecting the pivot, and if no pivot can be chosen from C, the branch (g, S, C) falls in the MissingTwoDeg problem instances and can be solved by `IRSolver`. Therefore, the existing branching strategy is not suitable for our framework as it will create some redundant branches.

Our branching strategy. Motivated by the above, we propose the following branching strategy called **BS-three**.

- **BS-three**. Given a branch (g, S, C), the pivot v_p is selected as the vertex in C that (1) has at least *three* non-neighbors in C and (2) has at least one non-neighbor in S; if no such vertex exists, the pivot v_p is chosen as an arbitrary vertex in C that has at least *three* non-neighbors in C.

We remark that if **BS-three** cannot find a pivot in C, the branch becomes a MissingTwoDeg problem and can be solved by `IRSolver`. In addition, **BS-three** cannot be applied to the previous BB framework [11, 12, 17] since they cannot handle the MissingTwoDeg problems (when no pivot can be found in C).

Time complexity of BBRes. With the proposed **BS-three**, our `BBRes` will generate $O(\lambda_k^{\delta\Delta})$ branches when $|V(g^*)| \geq k+1$, and generate $O(\lambda_k^n)$ branches otherwise in the worst case. Here, δ is the degeneracy of G, Δ is the maximum degree of G, and λ_k is the largest real root of the equation $x^{k+4} - 2x^{k+3} + x^3 - x + 1 = 0$ (based on the analytical method in [22]). For example, $\lambda_k = 1.381, 1.705, 1.867$ when $k = 1, 2, 3$. Thus, `BBRes` has the worst-case time complexity of $O^*(\lambda_k^{\delta\Delta})$ when $|V(g^*)| \geq k+1$, and of $O(\lambda_k^n)$ otherwise, where O^* suppresses the polynomials, as summarized in Lemma 4.3. Recall that the state-of-the-art method following Algorithm 1 has the time complexity of $O^*(\gamma_k^n)$ where γ_k is the largest real root of $x^{k+3} - 2x^{k+2} + x^2 - x + 1 = 0$, e.g., $\gamma_k = 1.466, 1.755, 1.889$ for $k = 1, 2, 3$. We remark that λ_k is *strictly smaller* than γ_k for $k \geq 1$.

Lemma 4.3. *Given a graph G and a integer k,* `BBRes` *runs in $O(n(\Delta\delta)^2\lambda_k^{\Delta\delta})$ when the largest k-defective clique in G is of size larger than $k+1$ (i.e., $|V(g^*)| \geq k+2$), and runs in $O(n^2\lambda_k^n)$, where δ is the degeneracy of G, Δ is the maximum degree of G, and λ_k is the largest real root of the equation $x^{k+4} - 2x^{k+3} + x^3 - x + 1 = 0$.*

Proof. We note that each recursion of `BBRes` runs in polynomial time $O(|C|^2)$, which is dominated by computing the upper bound (details are discussed in Section 5). When $|V(g^*)| \geq k+2$, we can utilize the diameter-two property, then the size of G_{v_i} can be bounded by $\Delta\delta$ [12] and thus $|C| \leq \Delta\delta$; otherwise, we have $|C| \leq n$. We then analyze the number of recursions (or branches) and put the details in Appendix A of our technical report [1] for simplicity. □

5 DOUBLE-COLORING UPPER BOUND

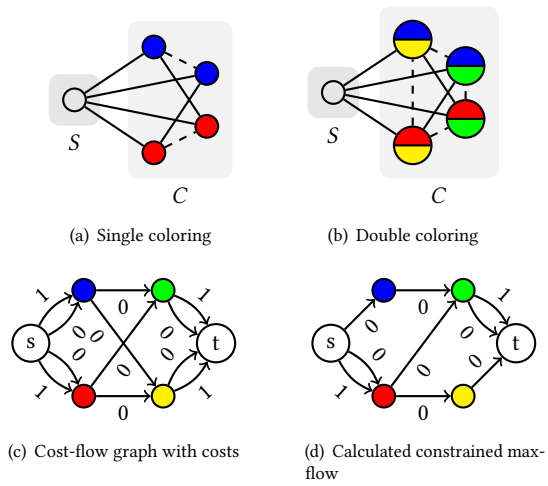
The upper bound estimation is a crucial component in algorithms for the maximum k-defective problem, as it directly impacts the efficiency and effectiveness of the BB search process. Existing state-of-the-art methods typically rely on graph coloring techniques [11, 12, 17]. By assigning colors to vertices in the candidate set, these methods partition the candidate set into several independent sets, thereby facilitating the computation of upper bounds. However, this classic approach makes a simplifying assumption: it treats every pair of vertices in different color classes as if they must be connected by an edge. While this assumption simplifies the analysis, it often leads to loose upper bounds, especially in graphs where the connectivity between color classes is sparse.

Recognizing this limitation, we propose to advance upper bound computation by introducing a novel *double-coloring* strategy in this section. Rather than relying on a single coloring, we assign each vertex a pair of colors, derived from two independent coloring processes. This dual perspective enables us to capture more refined relationships between vertices. For example, if two vertices fall into different color classes in the first coloring but share the same color in the second, we gain additional insight: despite what the first coloring suggests, these vertices actually belong to an independent set under the second coloring. To illustrate, consider the example in Figure 3, which contrasts the granularity of structural information captured by the two schemes. As shown in Figure 3(a), single coloring assigns identical colors to non-adjacent vertices (e.g., the blue nodes), implicitly treating them as a clique and thus identifying only 2 missing edges. In contrast, Figure 3(b) demonstrates that double coloring distinguishes these vertices via unique color pairs, successfully exposing all 4 missing edges (indicated by dashed lines). In the following, we first present the computation of the double-coloring upper bound (Section 5.1) and then prove its correctness (Section 5.2).

5.1 Double-Coloring Upper Bound Computation

Additional notations. We now leverage the richer structural information revealed by double coloring. Specifically, for each vertex v in the candidate set C, we assign a pair of colors, $(col_1(v), col_2(v))$, where $col_1(\cdot)$ and $col_2(\cdot)$ are two different vertex colorings. Both colorings respect the graph's edges: for any edge (u, v) in G, we ensure $col_1(u) \neq col_1(v)$ and $col_2(u) \neq col_2(v)$. We refer to each ordered pair $(col_1(v), col_2(v))$ as the *color pair* of v. The specific procedures for generating these colorings will be introduced later.

We further introduce a *double-coloring indicator* function $I_G(u, v)$, which is defined as 1 if two distinct vertices u and v share the same color in either of the two colorings, and 0 otherwise. In other words, $I_G(u, v)$ indicates whether u and v are placed in the same color class in at least one of the two independently obtained colorings. It is



(a) Single coloring (b) Double coloring

(c) Cost-flow graph with costs (d) Calculated constrained max-flow

Figure 3: Comparison of coloring strategies and upper bound computation ($k = 2$). (a) Single coloring misses non-edges. (b) Double coloring reveals them via unique color pairs. (c) Cost-flow network with edge costs. (d) Max flow of 3 with cost 2 (flow shown on edges), yielding UB = $|S| + 3$.

easy to observe that, for any induced subgraph g, the quantity $\frac{1}{2}\sum_{u,v\in V(g)} I_g(u,v)$ provides a lower bound on the number of missing edges in g, i.e., $\frac{1}{2}\sum_{u,v\in V(g)} I_g(u,v) \leq |\overline{E}(g)|$, where $|\overline{E}(g)|$ is the number of non-edges in g. This is because $I_g(u,v) = 1$ implies that u and v are not adjacent in g, but the converse may not hold.

Overview of double-coloring upper-bound computation. Given a branch (g, S, C), our goal is to leverage the structural information provided by double coloring to select a subset D of the candidate set C such that the size of the maximum k-defective clique g' in this branch (i.e., $S \subseteq V(g') \subseteq S \cup C$) is at most $|D| + |S|$. In other words, $|D| + |S|$ serves as our double-coloring upper bound, denoted as **UB-Double**. However, this is a non-trivial task, as incorporating information from two colorings introduces additional dependencies that must be carefully managed.

To compute **UB-Double**, we construct a corresponding *cost-flow graph* instance based on the current branch (g, S, C) and employ the classic *constrained maximum flow algorithm* [3, 19, 25]. In this setting, the objective is to maximize the flow (which corresponds to the number of vertices added to S), while ensuring that the total cost (which corresponds to the number of non-edges) does not exceed a threshold, say $k - |\overline{E}(S)|$. After obtaining the maximum flow in the cost-flow graph, we show that it corresponds to a subset D of the candidate set C. Our **UB-Double** is then defined as $|S| + |D|$.

In the following, we present our double-coloring upper bound method **UB-Double** based on the cost-flow graph and constrained maximum flow computation.

Double-coloring upper bound via cost-flow graph. Given a branch (g, S, C), we aim to compute a tight upper bound on the size of the maximum k-defective clique by leveraging the structural information provided by double coloring. To this end, we first assign two colorings to the vertices in C: the first coloring, $col_1(\cdot)$, is assigned greedily according to the degeneracy ordering; the second coloring, $col_2(\cdot)$, is assigned randomly, with the additional constraint that no two vertices share the same color pair, i.e., $(col_1(u), col_2(u)) = (col_1(v), col_2(v))$ only if $u = v$.

Based on the assigned colorings, we construct the *cost-flow (directed) graph* $g^c = (V^c, E^c)$. First, we define the vertex set V^c. Let $V_1' = \{col_1(v) \mid v \in C\}$ and $V_2' = \{col_2(v) \mid v \in C\}$ be the sets of colors derived from the two coloring processes. The vertex set is then defined as $V^c = \{s\} \cup \{t\} \cup V_1' \cup V_2'$. As in Figure 3(c) (with $k = 2$), this vertex set forms a layered topology: V_1' and V_2' constitute two distinct layers bridging the source s and the sink t. Next, we construct the edge set E^c to encode the vertex selection and the k-defective constraint. The construction rules are as follows:

- **Source edges:** For each vertex $u \in V_1'$, we add $k+1$ parallel edges from s to u, denoted as (s, u), with capacity 1 and costs $0, 1, \ldots, k$, respectively. These edges distribute the allowable missing-edge budget.
- **Sink edges:** Similarly, for each vertex $v \in V_2'$, we add $k + 1$ parallel edges from v to t, with capacity 1 and costs $0, 1, \ldots, k$, respectively.
- **Internal edges:** For each vertex $x \in C$ in the original graph, let $u = col_1(x)$ and $v = col_2(x)$. We add a directed edge (u, v) connecting the two color layers, with capacity 1 and cost $\overline{d}_S(x)$. This edge represents the potential selection of vertex x.
- **Reverse edges:** For every edge constructed above, we add a corresponding reverse edge with capacity 0 and the negation of the original cost.

Given the constructed cost-flow graph instance $g^c = (V^c, E^c)$, we solve the *constrained maximum flow problem* [3, 19, 25], which aims to maximize the flow such that the total cost does not exceed $k - |\overline{E}(S)|$. The value of the maximum flow obtained from this graph corresponds precisely to the size of the subset $D \subseteq C$ that can be added to S while satisfying the k-defective constraint. Our double-coloring upper bound, denoted as **UB-Double**, is then defined as $|S| + |D|$. As an illustration, Figure 3(d) presents the computation result for the running example (where $k = 2$). The algorithm identifies a valid flow of value 3 with a total cost of 2. Since the cost is within the budget, this implies a valid subset size $|D| = 3$, yielding a refined upper bound of $|S| + 3 = 4$. Furthermore, there exists a direct mapping between flows in g^c and the subset D. Specifically, if a unit of flow in the maximum flow solution passes through an edge (u, v) with $u \in V_1'$ and $v \in V_2'$, we include the corresponding vertex in the set D. To formalize this mapping, let $p : v \longrightarrow (col_1(v), col_2(v))$ denote the correspondence from g to g^c, where $v \in C$ and $(col_1(v), col_2(v)) \in E(g^c)$.

5.2 Correctness and Complexity of Our Double-Coloring Upper Bound

We now establish the correctness proof and analyze the time complexity of our double-coloring upper bound **UB-Double**.

Correctness proof. We first establish the correctness of our proposed **UB-Double**. Specifically, given any branch (g, S, C) and its corresponding cost-flow graph instance g^c, our goal is to show that for any k-defective clique g' in this branch (where $S \subseteq V(g') \subseteq S \cup C$), the size of g' cannot exceed $|S| + |D|$, where $D \subseteq C$ is the subset selected by our algorithm.

The central idea of our proof is to relate the number of missing edges in the induced subgraph $G[S \cup D]$ to the cost of the constrained maximum flow in the cost-flow graph. Specifically, we observe that the total number of missing edges in $G[S \cup D]$ is equal to the sum of three terms: $|\overline{E}(S \cup D)| = |\overline{E}(S)| + |\overline{E}(D)| + \sum_{u \in D} \overline{d}_S(u)$. Our proof proceeds in two main steps:

- First, as shown in Lemma 5.1, for any subset D, the double-coloring indicator provides a lower bound on the number of missing edges within D, i.e., $\frac{1}{2} \sum_{u,v \in D} I_G(u,v) \leq |\overline{E}(D)|$.
- Second, as established in Lemma 5.2, the cost $cost(g^c)$ computed in the cost-flow graph g^c exactly matches the sum of the double-coloring indicator over D and the number of missing edges between S and D, i.e., $cost(g^c) = \frac{1}{2} \sum_{u,v \in D} I_G(u,v) + \sum_{u \in D} \overline{d}_S(u)$.

Putting these observations together, we see that if the total cost in the cost-flow graph does not exceed $k - \overline{E}(S)$, then the total number of missing edges in any k-defective clique formed by S and D is at most k. This means that any k-defective clique can be represented as a feasible flow in our construction. Since our constrained maximum flow algorithm finds the largest possible D under the cost constraint, the solution $|S| + |D|$ serves as a valid upper bound on the size of any k-defective clique containing S in this branch. Thus, our **UB-Double** produces a correct upper bound.

We now provide the detailed proof. We begin by presenting a lemma that follows directly from the definition of the double-coloring indicator.

Lemma 5.1. *Given a graph $G = (V, E)$ and any subset $D \subseteq V$, we have $\frac{1}{2} \sum_{u,v \in D} I_G(u,v) \leq |\overline{E}(D)|$.*

We are ready to give the following result.

Lemma 5.2. *Given a constructed cost-flow graph g^c and its constrained maximum flow, there exists a corresponding vertex set D in C such that $cost(g^c) = \frac{1}{2} \sum_{u,v \in D} I_G(u,v) + \sum_{u \in D} \overline{d}_S(u)$.*

Proof Sketch of Lemma 5.2. The proof relies on establishing a bijection between the valid flows in the cost-flow graph g^c and the subsets $D \subseteq C$. Specifically, any internal edge between the color layers V_1' and V_2' carrying a unit of flow corresponds uniquely to a selected vertex in D.

The correctness of the upper bound hinges on the decomposition of the total flow cost into two components:

- **Cost of edges ($V_1' \to V_2'$):** By construction, these edges carry costs equal to $\overline{d}_S(\cdot)$. Thus, their sum exactly captures the number of missing edges between the selected subset D and the partial solution S, i.e., $\sum_{u \in D} \overline{d}_S(u)$.
- **Cost of edges ($s \to V_1'$ and $V_2' \to t$):** The edges incident to s and t are assigned costs $0, 1, \ldots, k$. Consequently, if f vertices sharing the same color are selected, the minimum cost flow incurs a cost of $\sum_{i=0}^{f-1} i = \binom{f}{2}$, which counts the number of vertex pairs colliding on that color. Summing this over all color classes in both colorings yields $\frac{1}{2} \sum_{u,v \in D} I_G(u,v)$, which serves as a lower bound for the missing edges within D.

Combining these, the total cost strictly enforces that the number of missing edges in $S \cup D$ does not exceed k. The complete derivation is provided in Appendix A of our technical report [1]. □

Table 1: Number of solved instances by the algorithms with a 3-hour limit (best performers are highlighted in bold).

	Real-world graphs				
	BBRes	kDC2	MDC	DnBk	WODC
$k = 1$	**139**	137	138	135	138
$k = 3$	**139**	136	136	135	**139**
$k = 5$	**139**	136	131	134	**139**
$k = 10$	**135**	128	127	124	134
$k = 15$	**132**	126	121	119	130
$k = 20$	**129**	114	113	111	127

Time complexity analysis. Consider a branch (g, S, C). **UB-Double** first colors the vertices in C twice: the first coloring requires $O(|E(g)|)$ time [8], and the second coloring requires $O(|C| \times c) = O(|C|^2)$ time, where c is the number of colors for C and $c \leq |C|$. Constructing the corresponding cost-flow graph takes $O(ck) = O(|C|k)$ time. Finally, we apply a classical maximum flow algorithm [3, 19, 25] to solve the problem, which runs in $O(k|C|^2 \log |C|)$ time. Thus, the overall time complexity of **UB-Double** is $O(k|C|^2 \log |C|)$.

6 EXPERIMENTS

In this section, we conduct experiments to evaluate the performance of our proposed algorithm BBRes against four baselines.

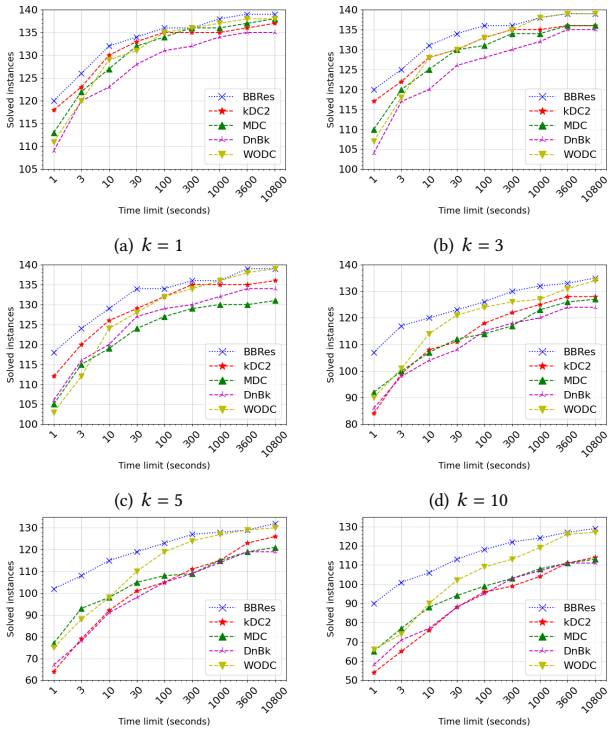
- kDC2 [1]: the existing algorithm proposed in [12].
- MDC [2]: the existing method proposed in [17].
- DnBK [3]: the existing algorithm proposed in [34], augmented with the same Stage II techniques (Algorithm 1) as kDC2 and MDC, for solving the maximum k-defective clique problem without the size constraint.
- WODC [4]: the state-of-the-art method proposed in [28].

We note that prior algorithms, including MADEC$^+$ [14], KDBB [23], KD-Club [29], and kDC [11], run significantly slower than the above baselines proposed recently and thus are omitted for the comparison. All algorithms are implemented in C++, compiled with -O3, and run on a machine with an Intel CPU @ 2.60GHz and 256GB main memory. Following previous studies [12, 17, 34], we choose k from $\{1, 3, 5, 10, 15, 20\}$ and set the time limit as 3 hours (i.e., 10,800s) for a fair comparison. We evaluate the algorithms on a widely adopted collection [5] of **139** real-world graphs [12, 17, 34] containing a total of 5.87×10^7 vertices. Our source code can be found in [1].

6.1 Comparison with State-of-the-Art Algorithms

Number of solved instances. Table 1 summarizes the number of instances solved by each algorithm within the 3-hour cutoff. First, we observe a general downward trend in the number of solved instances as k increases, confirming the exponential growth in computational complexity for larger k. Despite this challenge, BBRes consistently demonstrates superior performance, achieving the highest coverage across all settings. Significantly, the performance gap between BBRes and the competitors widens as k increases. For example, at $k = 20$, BBRes solves 129 instances, outperforming

[1] https://lijunchang.github.io/kDC-two/
[2] https://github.com/dawhc/MaximumDefectiveClique/
[3] https://github.com/cy-Luo000/Maximum-k-Defective-Clique/
[4] https://github.com/SNUCSE-CTA/WODC/
[5] http://lcs.ios.ac.cn/~caisw/Resource/realworld%20graphs.tar.gz



(a) $k = 1$ (b) $k = 3$ (c) $k = 5$ (d) $k = 10$ (e) $k = 15$ (f) $k = 20$

Figure 4: Number of solved instances with varying time limits

the runner-up (WODC) by 2 instances and significantly surpassing kDC2 by 15 instances. We attribute this advantage to the effective synergy between our diameter-2 partitioning and the proposed double-coloring upper bound. Real-world graphs typically exhibit a "globally sparse, locally dense" topology. While diameter-2 partitioning decomposes the globally sparse structure into smaller sub-instances, these sub-instances often retain high local density. It is in these dense regions that baseline methods falter, as their bounds (e.g., single-coloring) become loose, leading to redundant branches. In contrast, our double-coloring strategy captures finer-grained structural conflicts, thereby tightening the bounds and effectively pruning the search space in these dense substructures.

We extend our analysis to the time-efficiency trade-off by plotting the number of solved instances against the runtime budget (ranging from 1 second to 3 hours) in Figure 4. Across all tested values of k, BBRes maintains a dominant position, indicated by its curve consistently lying above those of the competitors (kDC2, MDC, DnBk, and WODC). Two key observations highlight the efficacy of BBRes: First, BBRes excels in rapid convergence. For hard instances where $k = 20$, BBRes quickly solves approximately 90 instances in the first second, whereas the baselines struggle to solve 65. Even with a generous 3-hour limit, BBRes reaches a total of 129 solved instances, surpassing the best baseline (WODC, 127 instances). Second, our method demonstrates superior robustness to problem difficulty. While the performance of baseline algorithms drops sharply as k increases (reflecting the exponential growth in search space), BBRes sustains a high completion rate. This suggests that our double-coloring bound effectively prunes the search space, mitigating the impact of increasing k.

Runtime efficiency on representative benchmarks. In this experiment, we evaluate the runtime performance of BBRes against four state-of-the-art algorithms, namely kDC2, MDC, DnBk, and WODC, on a subset of representative graphs with $k \in \{5, 10, 15, 20\}$. The dataset consists of the 30 largest graphs from the real-world collection, excluding instances where all algorithms exceeded the 3-hour time limit at $k = 20$. Table 3 reports the detailed running times. The results demonstrate that BBRes consistently delivers superior performance across all tested values of k, with the exception of a few trivial instances. Notably, on the soc-flixster graph with $k = 20$, BBRes achieves a remarkable 53× speedup over the best-performing baseline (WODC, 89.1s vs. 1.7s).

Furthermore, BBRes exhibits significant scalability and robustness with respect to increasing k. In contrast to the baselines, whose runtimes often degrade exponentially as k grows, BBRes remains highly efficient. For instance, on socfb-A-anon, as k increases from 10 to 20, the runtime of BBRes grows moderately from 7.8s to 17.1s. In stark contrast, the competitors suffer from severe performance degradation at $k = 20$: MDC, kDC2, and DnBk require 140.9s, 988.4s, and 1304.3s, respectively. Even the runner-up, WODC, is significantly slower, taking 57.3s, which is more than 3× that of BBRes. These results strongly validate the practical efficiency of our proposed algorithm in handling difficult constraints. Moreover, BBRes demonstrates exceptional capability in solving hard instances that are intractable for other methods. For example, on the graph sc-ldoor with $k = 20$, BBRes successfully finds the solution in 4,114 seconds, whereas all four baseline algorithms fail to terminate within the 3-hour cutoff. Similarly, on the large-scale graph soc-orkut ($k = 20$), only BBRes and WODC provide solutions, with BBRes being over 6× faster (146.8s vs. 892.6s). This evidence suggests that our double-coloring upper bound effectively prunes the search space even in the most computationally demanding scenarios.

6.2 Ablation Studies

To rigorously assess the individual contributions of our proposed techniques, we conduct an ablation study by comparing BBRes against two distinct variants. Specifically, we define -FlowUB as the variant that replaces our double-coloring upper bound (Section 5) with a standard single-coloring bound, thereby isolating the impact of our novel bounding strategy. Meanwhile, we define -Branch as the variant that disables our specific branching and early termination strategies (Section 4). We note that -Branch adopts the state-of-the-art branching strategy proposed in [17] for fair comparison, and it differs with MDC mainly in the upper bound. This comparison allows us to quantify the performance gain attributed solely to our optimized branching and early termination strategies.

Table 3 details the runtime performance of these methods across 30 representative benchmark graphs. The results consistently demonstrate that the full BBRes algorithm achieves superior performance compared to both variants in the vast majority of cases, validating the necessity of unifying both techniques. To quantify the impact of the double-coloring upper bound, we observe that -FlowUB suffers from significant performance degradation, particularly on large,

Table 2: Running time (in seconds) on 30 representative real-world benchmark graphs. '-' indicates timeout. The best performer is highlighted in bold; specifically, if the running time is within 10% of the fastest time, it is considered as the best.

			$k = 5$					$k = 10$					$k = 15$					$k = 20$				
Graphs	n	m	BBRes	kDC2	MDC	DnBk	WODC	BBRes	kDC2	MDC	DnBk	WODC	BBRes	kDC2	MDC	DnBk	WODC	BBRes	kDC2	MDC	DnBk	WODC
socfb-A-anon	3M	23M	**6.513**	**6.65**	19.343	21.89	**6.645**	**7.793**	37.453	32.101	48.902	8.919	**10.689**	289.987	52.543	305.076	18.556	**17.136**	988.404	140.895	1304.348	57.337
soc-orkut	2M	106M	**104.263**	157.448	137.127	390.314	405.808	**112.963**	576.496	188.372	523.604	455.723	**129.552**	8249.529	-	1353.479	573.453	**146.826**	-	-	-	892.649
socfb-B-anon	2M	20M	10.165	8.984	18.302	21.135	**6.883**	12.272	58.884	21.368	66.563	**7.622**	25.553	1737.898	45.803	778.94	**22.41**	**19.994**	8811.373	378.424	-	79.273
soc-flixster	2M	7M	**0.521**	4.136	2.74	2.461	1.189	**0.594**	38.122	3.875	8.435	1.871	**0.788**	1199.894	19.231	84.602	10.753	**1.677**	-	294.896	1785.488	89.086
web-wikipedia2009	1M	4M	**0.84**	**0.783**	2.083	1.289	1.872	0.954	**0.846**	2.779	2.551	2.542	**0.973**	**0.891**	2.889	49.022	2.726	**1.66**	**1.574**	3.085	16.88	2.596
tech-as-skitter	1M	11M	0.707	**0.593**	1.877	0.982	1.288	**0.755**	**0.702**	1.972	2.173	1.26	**0.886**	1.8	2.058	3.624	1.6	**1.04**	21.869	2.804	10.765	2.498
soc-pokec	1M	22M	5.996	**4.557**	11.098	16.999	7.88	6.588	**5.188**	14.35	17.144	7.634	**9.899**	**9.381**	18.949	101.537	**9.261**	**13.476**	58.022	23.248	430.095	24.827
soc-lastfm	1M	4M	3.392	105.634	88.744	104.585	**2.992**	**13.906**	2826.062	2098.176	-	**14.784**	**64.639**	-	-	-	**70.443**	**529.874**	-	-	-	**493.475**
soc-youtube-snap	1M	2M	**0.677**	3.844	2.42	3.009	1.247	**1.705**	56.95	28.339	31.979	3.735	**13.467**	1341.397	921.233	938.395	37.626	**193.191**	-	-	-	497.69
ca-hollywood-2009	1M	56M	**0.856**	**0.825**	3.263	**0.795**	3.36	0.854	**0.818**	3.315	**0.774**	3.35	**0.855**	**0.814**	3.364	**0.782**	3.174	**0.867**	**0.835**	3.401	**0.795**	3.15
sc-ldoor	952K	20M	**21.676**	64.183	47.185	-	494.426	**143.497**	1669.188	894.263	-	2290.9	**361.145**	3532.593	3093.435	-	4353.16	**4114.424**	-	-	-	-
soc-digg	770K	5M	**25.555**	38.424	43.787	1144.44	47.122	**33.257**	177.117	395.428	1885.92	61.119	**41.287**	8896.453	852.619	-	266.135	**64.569**	-	9918.25	-	3046.37
ca-coauthors-dblp	540K	15M	0.24	**0.205**	0.815	**0.224**	0.839	0.319	**0.245**	0.845	0.315	3.514	0.331	**0.258**	0.874	0.381	5.966	0.334	**0.269**	0.893	0.453	5.978
soc-delicious	536K	1M	0.075	**0.068**	0.437	0.151	0.169	**0.113**	0.135	0.417	0.295	0.301	**0.179**	0.391	0.587	0.686	0.474	**0.232**	4.095	1.537	2.157	1.037
web-it-2004	509K	7M	0.163	**0.14**	0.42	0.4	6.159	0.231	**0.202**	0.579	0.704	16.653	0.237	**0.214**	0.603	0.709	16.907	**0.243**	**0.225**	0.607	0.736	16.821
soc-youtube	495K	1M	**0.418**	1.619	1.28	1.953	1.123	**1.082**	31.709	16.115	17.539	4.14	**7.795**	758.326	-	509.977	41.827	**102.463**	-	-	-	674.757
sc-msdoor	415K	9M	**10.726**	31.332	24.22	1343.813	241.336	**81.904**	874.654	647.598	2487.434	1164.13	**199.684**	1897.013	2263.7	-	2211.78	**2360.887**	-	-	-	-
ca-MathSciNet	332K	820K	0.044	**0.032**	0.243	0.036	0.103	0.045	**0.032**	0.283	0.049	0.117	0.051	**0.038**	0.309	0.045	0.148	0.098	**0.08**	0.675	0.444	0.772
ca-dblp-2012	317K	1M	0.061	**0.043**	0.246	**0.043**	0.108	0.055	**0.045**	0.244	**0.047**	0.096	0.071	**0.047**	0.275	0.053	0.185	0.062	**0.037**	0.236	0.054	0.215
ca-citeseer	227K	814K	0.034	**0.026**	0.175	0.029	0.07	0.03	**0.021**	0.156	0.027	0.07	0.025	**0.021**	0.172	0.031	0.084	**0.029**	**0.028**	0.152	0.035	0.125
ca-dblp-2010	226K	716K	0.023	**0.018**	0.138	0.021	0.067	0.022	**0.019**	0.136	**0.02**	0.058	**0.023**	**0.025**	0.147	**0.023**	0.103	0.025	**0.019**	0.142	0.028	0.133
sc-pwtk	217K	5M	**1.679**	2.065	3.675	420.808	72.627	**5.801**	18.982	12.244	589.325	136.318	**32.956**	1544.762	180.301	1010.477	787.769	**133.684**	1655.622	451.792	1657.136	1771.97
soc-gowalla	196K	950K	**0.182**	0.219	0.314	0.241	**0.176**	**0.357**	1.409	0.406	0.489	**0.352**	**0.748**	67.38	0.831	1.937	**0.793**	6.406	2115.989	3.487	17.82	**2.709**
tech-RL-caida	190K	607K	**0.14**	**0.131**	0.176	0.178	0.17	**0.09**	0.179	0.188	0.361	0.339	**0.122**	1.873	1.075	1.282	0.833	**0.294**	14.532	-	10.435	2.249
sc-shipsec5	179K	2M	**0.176**	**0.161**	0.44	0.693	1.665	**0.258**	0.307	0.68	1.826	2.867	**0.884**	6.494	2.288	15.372	10.85	**2.344**	14.16	6.433	107.36	25.538
web-arabic-2005	163K	1M	**0.02**	**0.022**	0.12	0.023	0.088	**0.02**	0.023	0.119	0.024	0.088	**0.026**	**0.027**	0.123	**0.028**	0.145	**0.026**	**0.024**	0.121	**0.024**	0.146
soc-douban	154K	327K	**0.03**	**0.029**	0.117	0.109	0.09	**0.029**	**0.029**	4318.654	0.14	0.234	**7.509**	39.523	-	**7.028**	15.019	51.643	-	-	-	**5.31**
sc-shipsec1	140K	1M	0.14	**0.096**	0.216	0.13	0.337	**0.18**	**0.169**	0.624	0.927	1.812	**0.691**	**0.721**	1.677	3.125	5.079	**1.622**	6.267	28.64	34.677	17.956
web-uk-2005	129K	11M	0.252	**0.213**	0.472	0.65	17.792	0.304	**0.268**	0.575	0.839	25.944	0.316	**0.285**	0.595	1.002	26.144	**0.331**	**0.302**	0.616	1.088	26.286
web-sk-2005	121K	334K	**0.01**	**0.01**	0.055	**0.011**	0.132	**0.012**	**0.011**	0.066	0.014	0.208	**0.01**	**0.011**	0.056	0.015	0.213	0.016	**0.012**	0.067	0.016	0.217

Table 3: Running time (in seconds) of our algorithm BBRes and its variants on 30 representative benchmark graphs.

			$k = 5$			$k = 10$			$k = 15$			$k = 20$		
Graphs	n	m	BBRes	-FlowUB	-Branch	BBRes	-FlowUB	-Branch	BBRes	-FlowUB	-Branch	BBRes	-FlowUB	-Branch
socfb-A-anon	3M	23M	**6.092**	**6.214**	**6.045**	**7.128**	8.2	**7.036**	**9.75**	11.207	**10.429**	**17.047**	**18.637**	**17.456**
soc-orkut	2M	106M	**96.145**	106.618	**102.374**	**107.082**	120.143	**106.013**	**116.407**	175.704	**124.542**	**137.428**	314.834	159.698
socfb-B-anon	2M	20M	**10.3**	**10.631**	**10.507**	**12.518**	**12.193**	**11.913**	**20.905**	23.813	**21.743**	**21.171**	25.212	**21.966**
soc-flixster	2M	7M	**0.565**	1.044	**0.589**	**0.663**	1.652	**0.666**	**0.776**	4.169	0.958	**1.666**	23.701	4.299
web-wikipedia2009	1M	4M	**0.876**	**0.885**	**0.88**	**1.004**	**1.063**	1.134	0.924	**0.824**	**0.895**	**1.44**	**1.358**	**1.446**
tech-as-skitter	1M	11M	**0.64**	**0.662**	**0.648**	**0.766**	**0.748**	**0.773**	**0.902**	**0.919**	**0.922**	**1.118**	1.357	**1.145**
soc-pokec	1M	22M	**6.04**	6.287	**5.707**	**6.719**	**6.633**	**6.167**	8.94	**7.96**	**8.28**	**10.798**	**11.459**	**10.714**
soc-lastfm	1M	4M	**3.404**	13.886	**3.446**	**14.071**	63.493	**14.24**	**64.892**	238.069	**70.182**	**527.906**	1441.598	**545.187**
soc-youtube-snap	1M	2M	**0.674**	1.639	**0.692**	2.076	7.68	**1.721**	**13.386**	51.29	**13.649**	**191.974**	467.113	**190.072**
ca-hollywood-2009	1M	56M	**1.024**	**1.008**	**1.035**	**1.019**	**1.042**	**1.061**	**1.028**	**0.964**	**1.022**	**0.972**	**0.896**	**0.938**
sc-ldoor	952K	20M	**21.105**	23.411	**21.438**	140.731	**122.767**	167.192	**353.206**	**370.867**	497.15	-	-	-
soc-digg	770K	5M	**26.561**	**28.66**	**26.822**	**34.339**	40.286	**34.875**	**40.788**	62.653	**42.567**	**63.947**	280.828	75.775
ca-coauthors-dblp	540K	15M	**0.231**	**0.246**	**0.241**	**0.307**	**0.302**	**0.299**	**0.312**	**0.301**	**0.309**	**0.304**	**0.304**	**0.302**
soc-delicious	536K	1M	**0.098**	**0.101**	**0.102**	0.118	**0.105**	0.125	**0.146**	**0.15**	**0.15**	**0.238**	0.271	0.291
web-it-2004	509K	7M	**0.166**	**0.171**	**0.168**	**0.226**	**0.218**	**0.226**	**0.228**	**0.228**	**0.231**	**0.236**	**0.234**	**0.237**
soc-youtube	495K	1M	**0.402**	0.797	0.476	**1.093**	3.842	**1.085**	**7.682**	30.757	**7.793**	**101.615**	260.501	**101.357**
sc-msdoor	415K	9M	**10.503**	**11.31**	**10.774**	80.727	**65.374**	100.38	**197.788**	**196.372**	292.254	2322.411	**2055.29**	-
ca-MathSciNet	332K	820K	**0.059**	**0.059**	**0.06**	0.057	0.055	**0.045**	**0.046**	**0.049**	**0.05**	0.111	0.117	**0.089**
ca-dblp-2012	317K	1M	**0.05**	**0.052**	**0.053**	0.06	0.059	**0.05**	0.062	**0.051**	**0.053**	**0.042**	0.05	0.056
ca-citeseer	227K	814K	**0.031**	**0.034**	**0.032**	**0.031**	**0.032**	**0.03**	**0.032**	**0.033**	**0.03**	0.036	**0.027**	0.033
ca-dblp-2010	226K	716K	**0.023**	**0.024**	**0.023**	**0.024**	0.028	0.028	**0.022**	**0.022**	**0.024**	**0.022**	**0.022**	**0.022**
sc-pwtk	217K	5M	**1.602**	**1.612**	**1.62**	5.639	**4.65**	6.087	**32.577**	54.861	40.647	**130.557**	171.966	182.337
soc-gowalla	196K	950K	0.23	0.232	**0.181**	0.355	0.344	**0.279**	**0.729**	1.797	**0.743**	**6.432**	19.142	**6.753**
tech-RL-caida	190K	607K	**0.106**	0.134	**0.107**	**0.088**	0.108	0.106	**0.113**	0.154	**0.114**	**0.278**	0.497	**0.28**
sc-shipsec5	179K	2M	**0.17**	0.213	**0.176**	0.283	**0.245**	0.324	**0.878**	1.088	**0.89**	**2.272**	2.511	**2.391**
web-arabic-2005	163K	1M	**0.02**	**0.02**	**0.02**	**0.02**	0.024	0.024	**0.023**	0.028	**0.024**	**0.023**	**0.023**	**0.022**
soc-douban	154K	327K	**0.032**	**0.032**	**0.031**	0.039	0.038	**0.031**	**7.61**	**7.21**	**7.465**	**49.085**	**49.571**	**47.531**
sc-shipsec1	140K	1M	**0.107**	**0.106**	**0.111**	**0.173**	**0.173**	**0.177**	**0.573**	**0.571**	0.739	**1.316**	**1.312**	**1.389**
web-uk-2005	129K	11M	**0.252**	**0.257**	**0.258**	**0.307**	**0.305**	**0.308**	**0.318**	**0.316**	**0.316**	**0.33**	**0.328**	**0.325**
web-sk-2005	121K	334K	0.012	**0.01**	0.012	**0.013**	**0.013**	**0.013**	**0.01**	**0.01**	**0.01**	0.013	**0.01**	**0.01**

dense graphs with higher k. For instance, on `soc-lastfm` with $k = 20$, the runtime surges from 527.9 seconds (BBRes) to 1441.6 seconds (-FlowUB), representing a nearly 2.7× slowdown. Similarly, on `soc-orkut` ($k = 20$), -FlowUB requires 314.8 seconds compared to 137.4 seconds for BBRes. These results indicate that our double-coloring strategy provides a significantly tighter bound than the single-coloring approach, effectively pruning massive branches of the search tree that standard bounds fail to detect. Regarding the impact of the branching strategy, the comparison between BBRes and -Branch highlights the efficiency of our branching and termination rules. While -Branch generally performs better than -FlowUB (suggesting that the upper bound is often the dominant factor in

Table 4: Number of DIMACS instances solved by the algorithms with a 3-hour limit.

	DIMACS graphs				
	BBRes	kDC2	MDC	DnBk	WODC
$k = 1$	**45**	37	42	44	44
$k = 3$	**43**	23	31	19	41
$k = 5$	**40**	19	24	17	37
$k = 10$	**33**	14	18	16	**33**
$k = 15$	**24**	12	14	13	23
$k = 20$	**22**	11	14	12	16

runtime reduction), it still lags behind the full BBRes. For example, on sc-msdoor at $k = 15$, -Branch is approximately 1.5× slower than BBRes (292.3 seconds vs. 197.8 seconds). This confirms that our tailored branching strategy effectively guides the search towards high-quality solutions earlier, facilitating faster convergence. In summary, BBRes achieves superior robustness by leveraging the synergy between tight bounding and intelligent branching.

6.3 Performance on Dense and Synthetic Graphs

While the previous sections focused on sparse real-world graphs, the maximum k-defective clique problem is theoretically most challenging on *dense* graphs. To stress-test BBRes under these extreme conditions, we extended our evaluation to the DIMACS benchmark (notoriously hard dense instances) and LFR synthetic graphs [31] (controlled density analysis).

The results confirm that BBRes maintains its dominance even in high-density environments. As summarized in Table 4, BBRes consistently outperforms state-of-the-art baselines across all tested values of k. The performance gap becomes particularly pronounced for larger k; for instance, at $k = 20$, BBRes solves 22 instances, whereas kDC2 and DnBk only solve 11 and 12, respectively, marking a nearly 2× improvement in solvability. Furthermore, sensitivity analysis on LFR graphs reveals a positive correlation between graph density and our pruning efficiency: as density increases, the speedup of BBRes over its variants escalates significantly (up to 174×). This confirms that our double-coloring upper bound is particularly adept at pruning dense substructures. Due to space constraints, the detailed runtime breakdown for DIMACS instances and the complete density sensitivity analysis on LFR graphs are provided in Appendix B of the technical report [1].

7 RELATED WORK

The concept of the k-defective clique was originally formalized by [50]. Early exact methods for the maximum k-defective clique problem, such as those in [26, 27, 43], suffered from limited scalability and were practical only on small graphs. The first algorithm capable of handling large graphs was MADEC$^+$ [14]. Building on this foundation, KDBB [23] and KD-Club [29] improved empirical performance by combining preprocessing with several (upper-bound-based) pruning rules. Recently, several algorithms, including kDC [11], MDC [17], kDC2 [12], DnBK [34], and WODC [28], have introduced new branching strategies, thereby improving the theoretical time complexity of the problem. Furthermore, these methods introduce novel reduction rules and upper bound-based pruning rules to effectively eliminate unnecessary branches and vertices during the branch-and-bound search phase. In addition, they incorporate new heuristics to efficiently compute an initial solution in the preprocessing phase. We note that the most recent methods, namely kDC2 [12], MDC [17], DnBK [34], and WODC [28] are included in our experiments for comparison. From a theoretical perspective, MADEC$^+$ runs in $O^*(\beta_{2k}^n)$ time, while kDC further reduces the complexity to $O^*(\beta_k^n)$, where β_i is the largest real root of the equation $x^{i+3} - 2x^{i+2} + 1 = 0$. The kDC2 algorithm, proposed by Chang [12], runs in $O^*(\beta_{k-1}^n)$ time, and the MDC algorithm, proposed by Dai et al. [17], runs in $O^*(\gamma_k^n)$ time, where γ_i is the largest real root of the equation $x^{i+3} - 2x^{i+2} + x^2 - x + 1 = 0$. Importantly, we note that $\gamma_k < \beta_{k-1}$ for $k \geq 2$. Distinct from the above BB frameworks, DnBK and WODC employ alternative frameworks to reduce the exponential base. DnBK reduces the problem of finding the largest k-defective clique to $O(n(\Delta\delta)^{2k})$ instances of finding the maximum clique and runs in $O(n(\Delta\delta)^{2k}\alpha_c^n)$ time, where α_c is the base factor in the time complexity of the maximum clique algorithm. While theoretical bounds suggest $\alpha_c = 1.2$ [49], practical implementations rely on solvers like those in [10], where $\alpha_c \approx 1.33$ [6]. Similarly, Jang et al. [28] introduced WODC, which decomposes the branching process into a maximal clique enumeration stage and a k-defective clique constraint verification stage. Consequently, WODC has the worst-case time complexity of $O(n(\Delta\delta)^k\alpha_e^\delta)$, where $\alpha_e = 3^{1/3} \approx 1.44$ represents the optimal base for maximal clique enumeration. While DnBK and WODC achieve smaller exponential bases, they introduce significant overhead factors of $(\Delta\delta)^{\Theta(k)}$. These factors become prohibitive when k is not a small constant, limiting their scalability in general settings. In this paper, our BBRes achieves the time complexity of $O^*(\lambda_k^n)$, where λ_k is the largest real root of the equation $x^{k+4} - 2x^{k+3} + x^3 - x + 1 = 0$ and λ_k is strictly smaller than γ_k for $k \geq 1$. Thus, our proposed BBRes achieves better theoretical complexity for general k (noting that the complexity of DnBK and WODC depends exponentially on k), and empirically outperforms the SOTA methods, namely, kDC2, MDC, DnBK, and WODC, in Section 6.

In addition, the problem of enumerating all maximal k-defective cliques has been investigated. The Pivot+ algorithm [18] achieves $O^*(\gamma_k^n)$ time. However, Pivot+ is typically less effective for identifying the maximum k-defective clique in practice due to limited pruning techniques. Furthermore, the concept of defective cliques has been extended to bipartite graphs [16, 45].

8 CONCLUSION

In this paper, we proposed a new branch-and-bound method BBRes for finding the maximum k-defective clique. BBRes has the worst-case time complexity of $O^*(\lambda_k^n)$, which improves upon the state-of-the-art time complexity of $O^*(\gamma_k^n)$, where $\lambda_k < \gamma_k$. To boost the practical efficiency, we also proposed a tighter upper bound. Finally, extensive experiments demonstrate that BBRes outperforms state-of-the-art algorithms by several orders of magnitude in practice. In the future, we intend to investigate parallelization strategies for BBRes to handle massive graphs.

[6] As [10] omits the time complexity analysis for the maximum clique algorithm MC-BRB, we provide it here. MC-BRB recursively removes vertices with at most three non-neighbors in the candidate set. Thus, MC-BRB only branches on vertices with at least four non-neighbors in the candidate set. This leads to the characteristic equation $x^5 - x^4 - 1 = 0$, yielding a branching factor of $\alpha_c \approx 1.33$.

REFERENCES

[1] Technical Report and Source Code. https://github.com/Thaumaturge2020/BBRes/.

[2] Mohiuddin Ahmed, Abdun Naser Mahmood, and Md Rafiqul Islam. 2016. A survey of anomaly detection techniques in financial domain. *Future Generation Computer Systems* 55 (2016), 278–288.

[3] Ravindra K Ahuja and James B Orlin. 1995. A capacity scaling algorithm for the constrained maximum flow problem. *Networks* 25, 2 (1995), 89–98.

[4] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ algorithm for cores decomposition of networks. *CoRR* cs.DS/0310049 (2003).

[5] Punam Bedi and Chhavi Sharma. 2016. Community detection in social networks. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 6, 3 (2016), 115–135.

[6] Nina Berry, Teresa Ko, Tim Moy, Julienne Smrcka, Jessica Turnley, and Ben Wu. 2004. Emergent clique formation in terrorist recruitment. In *Proceedings of the AAAI Workshop on Agent Organizations: Theory and Practice*. 1198–1208.

[7] Jean-Marie Bourjolly, Gilbert Laporte, and Gilles Pesant. 2002. An exact algorithm for the maximum k-club problem in an undirected graph. *European Journal of Operational Research* 138, 1 (2002), 21–28.

[8] Daniel Brélaz. 1979. New methods to color the vertices of a graph. *Commun. ACM* 22, 4 (1979), 251–256.

[9] Randy Carraghan and Panos M Pardalos. 1990. An exact algorithm for the maximum clique problem. *Operations Research Letters* 9, 6 (1990), 375–382.

[10] Lijun Chang. 2020. Efficient maximum clique computation an enumeration over large sparse graphs. *The VLDB Journal* 29, 5 (2020), 999–1022.

[11] Lijun Chang. 2023. Efficient maximum k-defective clique computation with improved time complexity. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 3 (2023), 1–26.

[12] Lijun Chang. 2024. Maximum defective clique computation: Improved time complexities and practical performance. *Proceedings of the VLDB Endowment* 18, 2 (2024), 200–212.

[13] Lijun Chang, Mouyi Xu, and Darren Strash. 2022. Efficient maximum k-plex computation over large sparse graphs. *Proceedings of the VLDB Endowment* 16, 2 (2022), 127–139.

[14] Xiaoyu Chen, Yi Zhou, Jin-Kao Hao, and Mingyu Xiao. 2021. Computing maximum k-defective cliques in massive graphs. *Computers & Operations Research* 127 (2021), 105131.

[15] James Cheng, Yiping Ke, Ada Wai-Chee Fu, Jeffrey Xu Yu, and Linhong Zhu. 2011. Finding maximal cliques in massive networks. *ACM Transactions on Database Systems (TODS)* 36, 4 (2011), 1–34.

[16] Donghang Cui, Ronghua Li, Qiangqiang Dai, Hongchao Qin, and Guoren Wang. 2025. On the efficient discovery of maximum k-defective biclique. *arXiv preprint arXiv:2506.16121* (2025).

[17] Qiangqiang Dai, Ronghua Li, Donghang Cui, and Guoren Wang. 2024. Theoretically and practically efficient maximum defective clique search. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 4 (2024), 1–27.

[18] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023. Maximal defective clique enumeration. *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)* 1, 1 (2023), 1–26.

[19] Efim A Dinic. 1970. Algorithm for solution of a problem of maximum flow in networks with power estimation. *Soviet Math. Doklady* 11 (1970), 1277–1280.

[20] David Eppstein, Maarten Löffler, and Darren Strash. 2013. Listing all maximal cliques in large sparse real-world graphs. *Journal of Experimental Algorithmics (JEA)* 18 (2013), 3–1.

[21] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29 (2020), 353–392.

[22] Fedor V Fomin and Dieter Kratsch. 2010. *Exact Exponential Algorithms*. Springer Science & Business Media.

[23] Jian Gao, Zhenghang Xu, Ruizhi Li, and Minghao Yin. 2022. An exact algorithm with new upper bounds for the maximum k-defective clique problem in massive sparse graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 10174–10183.

[24] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, and Cheng Long. 2024. Maximum k-plex search: An alternated reduction-and-bound method. *Proceedings of the VLDB Endowment* 18, 2 (2024), 363–376.

[25] Andrew Goldberg and Robert Tarjan. 1987. Solving minimum-cost flow problems by successive approximation. In *Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*. 7–18.

[26] Timo Gschwind, Stefan Irnich, Fabio Furini, and Roberto Wolfler Calvo. 2021. A branch-and-price framework for decomposing graphs into relaxed cliques. *INFORMS Journal on Computing* 33, 3 (2021), 1070–1090.

[27] Timo Gschwind, Stefan Irnich, and Isabel Podlinski. 2018. Maximum weight relaxed cliques and Russian Doll Search revisited. *Discrete Applied Mathematics* 234 (2018), 131–138.

[28] Jihoon Jang, Yehyun Nam, Kunsoo Park, and Hyunjoon Kim. 2025. Efficient defective clique enumeration and search with worst-case optimal search space. *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)* 3, 6 (2025), 1–28.

[29] Mingming Jin, Jiongzhi Zheng, and Kun He. 2024. KD-Club: An efficient exact algorithm with new coloring-based upper bound for the maximum k-defective clique problem. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 20735–20742.

[30] Jalal Khalil, Da Yan, Guimu Guo, and Lyuheng Yuan. 2022. Parallel mining of large maximal quasi-cliques. *The VLDB Journal* 31, 4 (2022), 649–674.

[31] Andrea Lancichinetti, Santo Fortunato, and Filippo Radicchi. 2008. Benchmark graphs for testing community detection algorithms. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 78, 4 (2008), 046110.

[32] Kingsly Leung and Christopher Leckie. 2005. Unsupervised anomaly detection in network intrusion detection using clusters. In *Proceedings of the Australasian Conference on Computer Science*. 333–342.

[33] John M. Lewis and Mihalis Yannakakis. 1980. The node-deletion problem for hereditary properties is NP-complete. *J. Comput. System Sci.* 20, 2 (1980), 219–230.

[34] Chunyu Luo, Yi Zhou, Zhengren Wang, and Mingyu Xiao. 2024. A faster branching algorithm for the maximum k-defective clique problem. In *Proceedings of the European Conference on Artificial Intelligence (ECAI)*. 4132–4139.

[35] Panos M Pardalos and Jue Xue. 1994. The maximum clique problem. *Journal of global Optimization* 4, 3 (1994), 301–328.

[36] Bharath Pattabiraman, Md Mostofa Ali Patwary, Assefaw H Gebremedhin, Weikeng Liao, and Alok Choudhary. 2015. Fast algorithms for the maximum clique problem on massive graphs with applications to overlapping community detection. *Internet Mathematics* 11, 4-5 (2015), 421–448.

[37] Jeffrey Pattillo, Nataly Youssef, and Sergiy Butenko. 2013. On clique relaxation models in network analysis. *European Journal of Operational Research* 226, 1 (2013), 9–18.

[38] Jian Pei, Daxin Jiang, and Aidong Zhang. 2005. On mining cross-graph quasi-cliques. In *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (SIGKDD)*. 228–238.

[39] Pablo San Segundo, Alvaro Lopez, and Panos M Pardalos. 2016. A new exact maximum clique algorithm for large and massive sparse graphs. *Computers & Operations Research* 66 (2016), 81–94.

[40] Stephen B. Seidman and Brian L Foster. 1978. A graph-theoretic generalization of the clique concept. *Journal of Mathematical Sociology* 6, 1 (1978), 139–154.

[41] Etsuji Tomita. 2017. Efficient algorithms for finding maximum and maximal cliques and their applications. In *International workshop on algorithms and computation*. Springer, 3–15.

[42] Etsuji Tomita, Yoichi Sutani, Takanori Higashi, Shinya Takahashi, and Mitsuo Wakatsuki. 2010. A simple and faster branch-and-bound algorithm for finding a maximum clique. In *International Workshop on Algorithms and Computation*. Springer, 191–203.

[43] Svyatoslav Trukhanov, Chitra Balasubramaniam, Balabhaskar Balasundaram, and Sergiy Butenko. 2013. Algorithms for detecting optimal hereditary structures in graphs, with application to clique relaxations. *Computational Optimization and Applications* 56, 1 (2013), 113–130.

[44] Jianxin Wang, Min Li, Youping Deng, and Yi Pan. 2010. Recent advances in clustering methods for protein interaction networks. *BMC Genomics* 11, Suppl 3 (2010), S10.

[45] Zhiyi Wang, Lijun Chang, and Jeffrey Xu Yu. 2025. Identifying maximum defective bicliques in large bipartite graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 3710–3723.

[46] Zhengren Wang, Yi Zhou, Chunyu Luo, and Mingyu Xiao. 2023. A fast maximum k-plex algorithm parameterized by the degeneracy gap. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 5648–5656.

[47] Hongbo Xia, Kaiqiang Yu, Shengxin Liu, Cheng Long, and Xun Zhou. 2025. Maximum degree-based quasi-clique search via an iterative framework. In *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (SIGKDD)*. 3285–3296.

[48] Mingyu Xiao, Weibo Lin, Yuanshun Dai, and Yifeng Zeng. 2017. A fast algorithm to compute maximum k-plexes in social network analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 919–925.

[49] Mingyu Xiao and Hiroshi Nagamochi. 2017. Exact algorithms for maximum independent set. *Information and Computation* 255 (2017), 126–146.

[50] Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.

[51] Kaiqiang Yu and Cheng Long. 2021. Graph mining meets fake news detection. In *Data Science for Fake News: Surveys and Perspectives*. Springer, 169–189.

[52] Kaiqiang Yu and Cheng Long. 2023. Fast maximal quasi-clique enumeration: A pruning and branching co-design approach. *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)* 1, 3 (2023), 1–26.

[53] Zhiping Zeng, Jianyong Wang, Lizhu Zhou, and George Karypis. 2006. Coherent closed quasi-clique discovery from large dense graph databases. In *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (SIGKDD)*. 797–802.

[54] Yi Zhou, Shan Hu, Mingyu Xiao, and Zhang-Hua Fu. 2021. Improving maximum k-plex solver via second-order reduction and graph color bounding. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 12453–12460.

A OMITTED PROOFS

Proof of Lemma 4.3. We note that each recursion of BBRes runs in polynomial time $O(|C|^2)$, which is dominated by computing the upper bound and conducting the reductions (details are discussed in Section 5). When the size of the maximum k-defective clique $|V(g^*)| \geq k+2$, we can utilize the diameter-two property, where the size of G_{v_i} can be bounded by $\Delta\delta$ [12], implying $|C| \leq \Delta\delta$; otherwise, we have $|C| \leq n$. Below, we analyze the number of recursions (or branches).

We analyze the branching process based on the recursion tree $\mathcal{T}$ illustrated in Figure 5. Specifically, $\mathcal{T}$ is a binary tree where each node B_i represents a state (g_i, S_i, C_i). Recall that our branching strategy, **BS-three**, prioritizes selecting a pivot p_i that has at least one non-neighbor in the current partial set S_i. Based on this, the *left child* of B_i is generated by adding p_i to S_i (i.e., $S_{i+1} \leftarrow S_i \cup \{p_i\}$), while the *right child* corresponds to the branch where p_i is excluded from C_i.

We focus our analysis on the *leftmost path* $(B_0, B_1, \ldots, B_q)$, formed by recursively traversing the left children. We initially restrict our analysis to the scenario where S_0 is non-empty, deferring the discussion of the case where $S_0 = \emptyset$ to the end of the proof. We define the endpoint B_q as either a leaf node or the first node along this path satisfying $|C_q| \leq |C_{q-1}| - 2$. This definition implies that, in the transition to B_q, either 1) at least one additional vertex (besides the pivot) is pruned from C_{q-1} due to constraint violations (i.e., inability to form a k-defective clique with S_q); or 2) the remaining instance becomes tractable (i.e., it has fewer than three non-neighbors in C_{q-1} after removing p_{q-1}), allowing IRSolver to directly obtain the final solution (i.e., terminate the branch). We first analyze the depth of this leftmost path and prove that:

$$q \leq k. \tag{8}$$

To prove this, let B_x $(0 \leq x \leq q)$ denote the last node on the path such that the pivot p_x is fully connected to S_x (i.e., p_x introduces no missing edges). First, consider the case where no such node exists. This implies that every pivot p_i $(0 \leq i < q)$ along the path

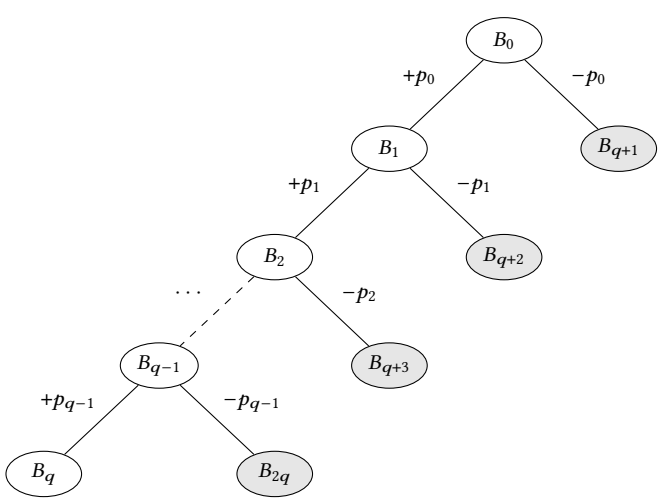


Figure 5: Branching Tree in the Proof of Lemma 4.3.

contributes at least one non-edge to S_{i+1}. Therefore, the number of missing edges grows monotonically: $|\overline{E}(S_{i+1})| \geq |\overline{E}(S_i)| + 1$. Summing over the path, we obtain $|\overline{E}(S_q)| \geq |\overline{E}(S_0)| + q$. Given the feasibility constraint that S_q is a k-defective clique (otherwise B_q would not be a valid branch), i.e., $|\overline{E}(S_q)| \leq k$, it follows that $q \leq k - |\overline{E}(S_0)| \leq k$. Next, we assume that such a node B_x exists. We claim that the accumulated number of missing edges in S_x satisfies $|\overline{E}(S_x)| > x$. Recall that B_q is defined as the first node where pruning occurs (i.e., $|C_q| \leq |C_{q-1}| - 2$). This implies that for any node B_i prior to B_q (specifically $0 \leq i < x$), no pruning occurred. Consequently, for each pivot $p_i \in \{p_0, \ldots, p_{x-1}\}$, its non-neighbors from C_i (at least three) must have been added to S_x. Since B_x is a node where p_x is fully connected to S_x, the candidate set C_x is cleared of any non-neighbors of S_x. Therefore, all non-neighbors of the set $\{p_0, \ldots, p_{x-1}\}$ must reside within S_x. In the complement graph of S_x, every vertex in $\{p_0, \ldots, p_{x-1}\}$ has a degree of at least 3. By the Handshaking Lemma, the total number of missing edges is at least the sum of degrees divided by 2, which implies that $|\overline{E}(S_x)| \geq \frac{3x}{2} = 1.5x$. For $x > 0$, we have $1.5x \geq x + 0.5 > x$. We now address the boundary case $x = 0$. If $|\overline{E}(S_0)| > 0$, the claim $|\overline{E}(S_x)| > x$ holds trivially since $|\overline{E}(S_0)| \geq 1 > 0$. If $|\overline{E}(S_0)| = 0$ (i.e., S_0 is a clique), we show that x cannot be 0. Note that S_0 contains an inherited pivot p' with at least three non-neighbors in C_0. If $x = 0$, B_0 would be the fully connected node B_x, implying that C_0 contains no non-neighbors of S_0 (including p'), which is a contradiction. Thus, if S_0 is a clique, we must have $x \geq 1$, ensuring the bound holds. Combining this with the monotonicity of missing edges after B_x, we derive $|\overline{E}(S_q)| \geq |\overline{E}(S_x)| + (q - x - 1) \geq (x+1) + q - x - 1 = q$. Given the feasibility constraint $|\overline{E}(S_q)| \leq k$, it follows that $q \leq k$.

Let $\ell(B_0)$ denote the number of leaf nodes rooted at node B_0 with non-empty S_0. We then prove that $\ell(B_0) \leq \lambda_k^{|C_0|}$ by induction. For the base case where B_0 is a leaf node, w.l.o.g., we let C_0 be empty since the corresponding branch can be solved directly. We thus have $\ell(B_0) = 1 \leq \lambda_k^{|C_0|}$. For a non-leaf node B_0, we consider the path $(B_0, B_1, ..., B_q)$ and trivially have $\ell(B_0) = \ell(B_q) + \ell(B_{q+1}) + ... + \ell(B_{2q})$, where B_{q+i} $(1 \leq i \leq q)$ is the right child of node B_{i-1} (See Figure 5). We have:

(1) $q \leq k$ based on the above discussion;
(2) $|C_q| \leq |C_{q-1}| - 2 \leq |C_0| - q - 1$ based on the definition of B_q;
(3) $|C_{q+i}| \leq |C_{i-1}| - 1 \leq |C_0| - i$ for $1 \leq i \leq q$, based on the branching relationship between the parent node B_{i-1} and its children B_i and B_{q+i}.

We note that when $q = k$, $\ell(B_0)$ reaches the largest number of leaf nodes, which corresponds to the worst case (we will discuss other cases $q < k$ later). In addition, when $q = k$, we note that $|\overline{E}(S_k)| = k$ based on the above discussion of $q \leq k$. Therefore, we have:

$$|C_k| \leq |C_{k-1}| - 4 \leq |C_0| - k - 3. \tag{9}$$

This is because p_{k-1} has three non-neighbors in C_{k-1} and those non-neighbors can be pruned since each of them cannot form a k-defective clique with S_k. We thus have:

$$\begin{aligned} \ell(B_0) &= \ell(B_q) + \ell(B_{q+1}) + ... + \ell(B_{2q}) && (10)\\ &\leq \ell(B_k) + \ell(B_{k+1}) + ... + \ell(B_{2k}) && (11)\\ &\leq \lambda_k^{|C_0|-k-3} + \lambda_k^{|C_0|-1} + ... + \lambda_k^{|C_0|-k}, && (12) \end{aligned}$$

where the inequality of $\lambda_k^{|C_0|-k-3} + \lambda_k^{|C_0|-1} + ... + \lambda_k^{|C_0|-k} \leq \lambda_k^{|C_0|}$ holds if λ_k is the largest real root of the equation $x^{k+4} - 2x^{k+3} + x^3 - x + 1 = 0$.

When $q < k$, the recurrence becomes:

$$\begin{aligned} \ell(B_0) &= \ell(B_q) + \ell(B_{q+1}) + ... + \ell(B_{2q}) && (13) \\ &\leq \lambda_k'^{|C_0|-q-1} + \lambda_k'^{|C_0|-1} + ... + \lambda_k'^{|C_0|-q}, && (14) \end{aligned}$$

where λ_q' is the largest real root of the equation $x^{q+2} - 2x^{q+1} + 1 = 0$. Since λ_q' demonstrably smaller than λ_k for $q < k$, we adopt λ_k as our worst-case time complexity factor.

Finally, we consider a general branch B_0 without the constraint $S_0 \neq \emptyset$. Similar to the proof of Lemma 4.3 in [12], we can prove that the number of leaf nodes satisfies $\ell(B_0) \leq \lambda_k^{|C_0|}$, where $|C_0| = |V(g_0)|$. If the size of the maximum k-defective clique is at least $k+2$, we can apply the diameter-two property to bound the initial candidate set by $|C_0| \leq \Delta\delta$ [12], yielding a time complexity of $O^*(\lambda_k^{\Delta\delta})$. Otherwise, we trivially have $|C_0| \leq n$, yielding $O^*(\lambda_k^n)$. □

Proof of Lemma 5.2. We first show that there is a one-to-one correspondence between the selected flow edges in the constrained maximum flow of g^c and the set D.

Recall that each vertex $v \in C$ in g corresponds to an edge $(col_1(v), col_2(v)) \in E(g^c)$ via the mapping $p : v \mapsto (col_1(v), col_2(v))$. The capacity of each edge between partitions V_1' and V_2' is set to 1, so in the maximum flow, each such edge either carries 1 unit of flow (selected) or 0 (not selected). Define E_{ans}^c as the set of edges with flow value 1; this set corresponds to a subset D of C, with $|E_{\text{ans}}^c| = flow(g^c)$. Let $f_{\text{out}}(v)$ and $f_{\text{in}}(v)$ be the outgoing and incoming flow at vertex v in g^c. Since flow is conserved at each vertex except for s and t, we have $f_{\text{in}}(v) = f_{\text{out}}(v)$. Thus, $|E_{\text{ans}}^c| = \sum_{u \in V_1'} f_{\text{out}}(u) = \sum_{u \in V_1'} f_{\text{in}}(u) = f_{\text{out}}(s) = flow(g^c)$. This establishes the mapping between the selected flow edges in the constrained maximum flow and the set D.

Now, we analyze the cost $cost(g^c)$ associated with the constrained maximum flow in g^c, which corresponds to the sum of the costs of all flow-carrying edges. We partition the total cost as $cost(g^c) = cost(E_1^c) + cost(E_2^c) + cost(E_3^c)$, where E_1^c denotes the edges between the source s and the first color class V_1', E_2^c denotes the edges between the second color class V_2' and the sink t, and E_3^c denotes the edges between V_1' and V_2' in g^c.

First, we have

$$cost(E_3^c) = \sum_{e \in E_{\text{ans}}^c} cost(e) = \sum_{e \in E_{\text{ans}}^c} \overline{d}_S(p^{-1}(e)) = \sum_{u \in D} \overline{d}_S(u),$$

where the second equality follows the construction of g_c and the mapping function p, and the last equality follows from the correspondence between E_{ans}^c and D.

Then, we consider $cost(E_1^c) + cost(E_2^c)$. For each vertex v in V_1', the flow is $f_{\text{in}}(v)$ which implies that there are $f_{\text{in}}(v)$ selected edges in the maximum flow between s and v. By our construction of g^c and the constrained maximum flow, the $f_{\text{in}}(v)$ edges with minimum cost, i.e., those corresponding to $\{0, 1, \ldots, f_{\text{in}}(v) - 1\}$, will be selected. The same observation applies to each vertex v in V_2'. Thus, we have $cost(E_1^c) = \sum_{v \in V_1'} \sum_{i=0}^{f_{\text{in}}(v)-1} i$ and $cost(E_2^c) = \sum_{v \in V_2'} \sum_{i=0}^{f_{\text{out}}(v)-1} i$.

Subsequently, we can derive the following:

$$\begin{aligned}
&\sum_{v \in V_1'} \sum_{i=0}^{f_{\text{in}}(v)-1} i + \sum_{v \in V_2'} \sum_{i=0}^{f_{\text{out}}(v)-1} i \\
&= \frac{1}{2}\left(\sum_{v \in V_1'} f_{\text{in}}^2(v) - f_{\text{in}}(v)\right) + \frac{1}{2}\left(\sum_{v \in V_2'} f_{\text{out}}^2(v) - f_{\text{out}}(v)\right) \\
&= \frac{1}{2}\left(\sum_{v \in V_1'} \sum_{\substack{p \in D \\ \text{col}_1(p)=v}} \sum_{\substack{q \in D \\ \text{col}_1(q)=v}} 1 - \sum_{v \in V_1'} \sum_{\substack{p \in D \\ \text{col}_1(p)=v}} 1\right) \\
&\quad + \frac{1}{2}\left(\sum_{v \in V_2'} \sum_{\substack{p \in D \\ \text{col}_2(p)=v}} \sum_{\substack{q \in D \\ \text{col}_2(q)=v}} 1 - \sum_{v \in V_2'} \sum_{\substack{p \in D \\ \text{col}_2(p)=v}} 1\right) \\
&= \frac{1}{2}\left(\sum_{p \in D} \sum_{q \in D} [\text{col}_1(p) = \text{col}_1(q)] - \sum_{p \in D} 1\right) \\
&\quad + \frac{1}{2}\left(\sum_{p \in D} \sum_{q \in D} [\text{col}_2(p) = \text{col}_2(q)] - \sum_{p \in D} 1\right) \\
&= \frac{1}{2}\left(\sum_{p \in D} \sum_{q \in D} [\text{col}_1(p) = \text{col}_1(q)] + [\text{col}_2(p) = \text{col}_2(q)] - 2\sum_{p \in D} 1\right) \\
&= \frac{1}{2} \sum_{u,v \in D} I_G(u,v),
\end{aligned}$$

where the second equality follows from the fact that $f_{\text{in}}(v) = \sum_{p \in D \text{ and } \text{col}_1(p)=v} 1$, and the last equality follows directly from the definition of the double-coloring indicator.

Combining the above, we have:

$$\begin{aligned}
cost(g^c) &= cost(E_1^c) + cost(E_2^c) + cost(E_3^c) \\
&= \sum_{c \in V_1'} \sum_{i=0}^{f_{\text{in}}(c)-1} i + \sum_{c \in V_2'} \sum_{i=0}^{f_{\text{out}}(c)-1} i + \sum_{e \in E_{\text{ans}}^c} cost(e) \\
&= \frac{1}{2} \sum_{u \in D} \sum_{v \in D} I_G(u,v) + \sum_{u \in D} \overline{d}_S(u).
\end{aligned}$$

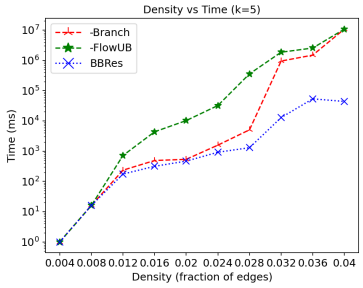
This completes the proof of Lemma 5.2. □

B ADDITIONAL EXPERIMENTS ON DENSE AND SYNTHETIC GRAPHS

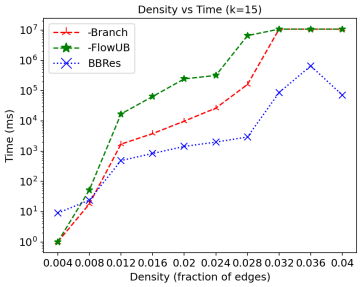
While the experimental results in the main text demonstrate the superior efficiency of BBRes on large-scale real-world graphs (which are typically sparse), the maximum k-defective clique problem is theoretically most challenging on **dense graphs** due to the exponential explosion of the search space. To rigorously stress-test our algorithm under these extreme conditions and to verify the effectiveness of our pruning strategy in high-density environments, we extend our evaluation to two complementary datasets:

Table 5: Running time (in seconds) on 10 representative DIMACS benchmark graphs. The best performer is highlighted in bold; specifically, if the running time is within 10% of the fastest time, it is considered as the best.

			$k = 5$					$k = 10$					$k = 15$					$k = 20$				
Graphs	n	m	BBRes	kDC2	MDC	DnBk	WODC	BBRes	kDC2	MDC	DnBk	WODC	BBRes	kDC2	MDC	DnBk	WODC	BBRes	kDC2	MDC	DnBk	WODC
c-fat500-2	500	9K	**0.001**	0.002	0.005	0.003	0.083	**0.001**	0.004	0.008	0.006	0.084	**0.01**	0.046	0.045	0.145	0.388	**0.045**	0.096	0.072	0.339	0.514
c-fat500-5	500	23K	**0.004**	0.011	0.014	0.008	0.397	**0.009**	0.023	0.023	0.014	0.383	**0.011**	0.034	0.032	0.025	0.577	**0.015**	0.043	0.044	0.028	0.661
c-fat500-1	500	4K	**0.0**	**0.0**	0.003	0.002	0.029	0.006	**0.005**	0.007	0.023	0.056	0.009	**0.007**	0.017	0.024	0.071	**1.196**	6.744	11.956	220.672	**1.282**
c-fat500-10	500	46K	**0.022**	0.055	0.035	0.028	1.206	**0.029**	0.117	0.056	0.074	1.722	**0.029**	0.18	0.116	0.093	1.858	**0.047**	0.242	0.134	0.098	2.201
san400-0-5-1	400	39K	56.024	2070.312	2632.115	-	**9.666**	**93.256**	-	7391.9	-	6707.4	**95.123**	-	-	-	5638.29	**460.396**	-	-	-	-
MANN-a27	378	70K	**2262.145**	-	3057.384	-	-	**2319.961**	-	-	-	-	**2265.974**	-	-	-	-	**3280.755**	-	-	-	-
hamming8-2	256	31K	**0.601**	1827.654	787.899	-	111.738	**17.051**	-	-	-	1328.32	**339.576**	-	-	-	-	**2109.191**	-	-	-	-
san200-0-7-1	200	13K	14.743	2509.822	72.747	205.319	**0.917**	74.559	-	1323.845	1202.407	**13.874**	**83.843**	-	-	-	237.672	**101.182**	-	-	-	-
c-fat200-2	200	3K	0.005	**0.001**	0.002	**0.001**	0.028	0.006	**0.002**	**0.002**	**0.002**	0.037	**0.005**	0.015	0.025	0.036	0.089	**0.015**	0.035	0.021	0.086	0.22
c-fat200-5	200	8K	0.003	0.005	0.005	**0.002**	0.146	**0.004**	0.015	0.008	0.005	0.204	**0.006**	0.016	0.013	**0.006**	0.259	**0.008**	0.027	0.016	0.01	0.319



(a) $k = 5$



(b) $k = 15$

Figure 6: Runtime analysis on LFR synthetic graphs with varying densities. The results illustrate the scalability of BBRes compared to its variants as graph density increases ($k = 5$ and $k = 15$).

- **DIMACS Benchmarks**[7]: A standard collection comprising **78** dense graphs. These instances serve as a stress test for raw performance on notoriously hard combinatorial problems.
- **LFR Synthetic Graphs**: Generated via the LFR benchmark [31] with $n = 5,000$. By systematically varying the graph density from 0.004 to 0.04, these graphs enable a fine-grained sensitivity analysis of the algorithms to varying graph densities.

Detailed performance on DIMACS benchmarks. While Section 6.3 presented the aggregate number of solved instances (Table 4), Table 5 in this appendix provides a granular breakdown of the running times on 10 representative DIMACS graphs. These detailed statistics offer a deeper insight into the performance gaps between BBRes and the baselines, particularly on hard instances. The results in Table 5 reveal that BBRes achieves orders-of-magnitude speedups. Taking the hamming8-2 graph as an example, for $k \in \{5, 10, 15\}$, BBRes finishes in 0.6s, 17.1s, and 339.6s, respectively. In stark contrast, kDC2, MDC, and DnBk require over 700s even for the easier case of $k = 5$ and fail to terminate within the 3-hour cutoff (10,800s) for larger k. WODC also struggles, taking 111.7s for $k = 5$ (186× slower than BBRes) and timing out for $k \geq 10$. We attribute this efficiency to the synergy between our diameter-2 partitioning and the double-coloring upper bound, which effectively prunes the search space in the dense clusters characteristic of DIMACS graphs.

Impact of graph density. To explicitly isolate the correlation between graph density and our pruning efficiency, we evaluate BBRes and its ablation variants (-Branch and -FlowUB) on the LFR synthetic datasets. Figure 6 plots the runtime against varying graph densities for $k = 5$ and $k = 15$. The results clearly demonstrate that as density increases, the performance advantage of BBRes becomes increasingly pronounced. Specifically, while BBRes outperforms its variants across the board, the relative speedup ratio grows substantially with density. For instance, at $k = 5$ (Figure 6(a)), on the graph with a density of 0.012, -Branch and -FlowUB are 1.3× and 4.1× slower than BBRes, respectively. However, at a higher density of 0.032, these slowdown factors escalate to approximately 73× and 174×. This empirical evidence confirms that our double-coloring upper bound is particularly effective in handling high-density regions, where simpler bounds (such as those used in -FlowUB) become loose and ineffective.

[7] https://networkrepository.com/dimacs.php